

Contents

Take-Home Assignment #2	2
Part I	2
1. Expected Shortfall and Value at Risk Estimation with GAt as Data Generating Process	2
2. Expected Shortfall and Value at Risk Estimation with Student's t Distribution as Data Generating Process	6
3. Expected Shortfall and Value at Risk Estimation with True DGP as Asymmetric Stable Paretian Distribution	6
3. Bonus	9
Part II	11
1. Simulation of a Bivariate Discrete Mixture of Laplace Distributions for Financial Returns Modeling	11
2. Maximum Likelihood Estimation for Simulated Mixture Models and Parameter Comparison	14
2a. Verification of the Modified Bessel Function of the Third Kind through Numerical Integration	14
2b. Maximum Likelihood Estimation for a Bivariate Laplace Mixture Using BFGS Optimization	15
3. MLE Computation for Bivariate NCT and Mixture of Bivariate Laplace Distributions .	18
4. Model Comparison of Bivariate Mixture-Laplace and NCT Distributions on DJIA Stock Pairs: Empirical Application	22
Appendices	26
A MATLAB Code for Part I Task 1: GAt Quantile Function	26
B MATLAB Code for Part I Task 1: ES and VaR for all Distributions	27
C MATLAB Code for Part I Task 1: MLE for all used Distributions	29
D MATLAB Code for Task I Task 3: ES and VaR for Asymmetric Stable Paretian . .	33
E MATLAB Code for Part I Bonus: MLE for Asymmetric Stable Paretian	34
F MATLAB Code for Task II Task 1: Mixture-density and Sample Data for Bivariate Laplace	37
G MATLAB Code for Task II Task 2: Bessel Function and MLE for discrete mixture of bivariate Laplace	38
H MATLAB Code for Task II Task 2b	39
I MATLAB Code for Task II Task 3: MLE for Bivariate NCT	40
J MATLAB Code for Task II Task 4: Pair Selection and MLE in DJIA Data	43

Take-Home Assignment #2

As in Take-Home Assignment #1, all solutions were implemented using MATLAB. The reader can find the full comprehensive code attached in the appendix. For structural simplicity, the code provided in the report body does not contain any comments or plot functions sometimes leaving out certain parts of the code. It is solely for the reader to follow the intuition of the results provided while reading.

Part I

1. Expected Shortfall and Value at Risk Estimation with GAt as Data Generating Process

In this part of the exercise we replicate Figure 4 from *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paoletta, 2009), focusing on the computation of Value-at-Risk (VaR) and Expected Shortfall (ES) for data generated from a Generalized Asymmetric t-distribution (GAt).

The density of the GAt distribution is given by:

$$f_{\text{GAt}}(z; d, \nu, \theta) = C_{d, \nu, \theta} \times \begin{cases} \left(1 + \frac{(-z \cdot \theta)^d}{\nu}\right)^{-(\nu + \frac{1}{d})}, & \text{if } z < 0, \\ \left(1 + \frac{(z/\theta)^d}{\nu}\right)^{-(\nu + \frac{1}{d})}, & \text{if } z \geq 0, \end{cases}$$

where $d, \nu, \theta \in \mathbb{R}_{>0}$, and

$$C_{d, \nu, \theta}^{-1} = (\theta^{-1} + \theta) d^{-1} \nu^{1/d} B\left(\frac{1}{d}, \nu\right)$$

The conditional expectation of Z^r where $Z \sim \text{GAt}$ is relevant for our bootstrap simulation study of the Expected Shortfall and is given by:

$$\mathbb{E}[Z^r \mid Z < c] = (-1)^r \nu^{r/d} \frac{(1 + \theta^2)}{(\theta^r + \theta^{r+2})} \frac{B_L\left(\nu - \frac{r}{d}, \frac{r+1}{d}\right)}{B_L\left(\nu, \frac{1}{d}\right)} \quad (1)$$

where

$$L = \frac{\nu}{\nu + (-c\theta)^d}$$

and B_L represents the incomplete beta function as derived in *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paoletta, 2009).

To generate independent and identically distributed (IID) samples from the GAt distribution we use Listing 3. In Figure 1 we then calculate the true Value at Risk and Expected Shortfall (given by the red line) and estimate the Expected Shortfall (ES) and Value-at-Risk (VaR) assuming the following distributions (parametric bootstrap): Gaussian (Normal) distribution, Student's t-distribution, Generalized Asymmetric t-distribution (GAt), Noncentral t-distribution (NCT), Two-component mixed normal distribution (Mixture of Normals) and apart from the parametric bootstrap also use the nonparametric bootstrap. Except for the nonparametric and mixed normal distributions, all other distributions are location-scale families.

Here we can also make of the fact that if Z is a location-zero, scale-one random variable and $Y = \sigma \cdot Z + \mu$ for $\sigma > 0$, then

$$ES(Y) = \mu + \sigma \cdot ES(Z)$$

and obviously (given a VaR)

$$ES(Z) = E[Z|Z < VaR]$$

To compute the true VaR and ES we use Listing 1 using the previously shown formal definition of the Expected Shortfall for the GAt in equation (1).

Listing 1: MATLAB Code for Computation of (True) Expected Shortfall for GAt

```

1 function ES = compute_ES_GA_t(d, v, theta, gamma)
2     VaR = GAtquantile(gamma, d, v, theta);
3     L = v / (v + (-VaR * theta)^d);
4     a_num = v - (1/d);
5     b_num = 2/d;
6     a_den = v;
7     b_den = 1/d;
8     B_L_num = betainc(L, a_num, b_num) * beta(a_num, b_num);
9     B_L_den = betainc(L, a_den, b_den) * beta(a_den, b_den);
10    ES = (((-1)*v^(1/d))*(1+theta^2)) / ((1 + theta^2) * theta)) * (B_L_num /
11        B_L_den);
end

```

To compute the VaR for the GAt we use Listing 2 which makes use of the quantile function, i.e. the evaluation of the CDF given a certain level γ . The quantile function is given in the appendix in Listing 26.

Listing 2: MATLAB Code for Computation of (True) VaR for GAt

```

1 VaR_values = mu_est + c_est * GAtquantile(gamma, d_est, v_est, theta_est);

```

So we program a function to input the parameters of the GAt distribution (location, scale, degrees of freedom, skewness and ES Level), where ESlevel is the number in (0,1) indicating the tail probability returning the true Expected Shortfall Value defined here as the tail probability γ . Since the ES is important to determine the potential "Loss" we multiply it by -1 as given in (1) and shown in Listing 1. Then we compute the PDF at the Value at Risk (VaR) for the GAt-distribution and henceforth can compute the Expected Shortfall for the GAt-distribution using the correct formula.

As mentioned, the Generalized Asymmetric t (GAt) distribution is a flexible distribution that can model skewness and kurtosis. In our GAt estimation function we use the MLE tailored specifically to the GAt distribution, including parameter bounds and constraints. The Noncentral t-Distribution (NCT) distribution is an extension of the Student's t-distribution that includes a noncentrality parameter to model asymmetry.

For the empirical part, we use the approach mentioned in *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paolella, 2009) which is computing the MLE of each of 100 resampled series. It appears (after trial and error) that a mixture of nonparametric and parametric bootstrap was done here. So sampling data from the GAt, then for each method resample 100 times and compute the MLE.

Again, to simulate sample data we run Listing 3 drawing from the PDF of the GAt.

Listing 3: MATLAB Code Sampling from GAt Distribution

```

1 function x = GAtsim(sim, d, v, theta)
2     x = zeros(sim,1);
3     lo = 1e-6; hi = 1 - lo;
4     for i = 1:sim
5         u = rand;
6         u = max(u, lo);
7         u = min(u, hi);
8         x(i) = GAtquantile(u, d, v, theta);
9     end
10 end

```

Figure 1 shows our results. Since the data is generated from the GAt distribution, methods that can capture skewness and heavy tails should perform better. The Non-parametric bootstrap performs well quite well for VaR and ES estimation in this sample, the GAt parametric bootstrap clearly performs best which was to be expected since the data generating process (DGP) is the GAt although the non-parametric bootstrap is closer in mean to the true VaR and ES, it's variation around the mean is much bigger than the one for the GAt parametric bootstrap. Some variation is expected due to sampling.

The Gaussian method, being symmetric and light-tailed, overestimates the risk measures which is in inline with Figure 4 in *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paolella, 2009).

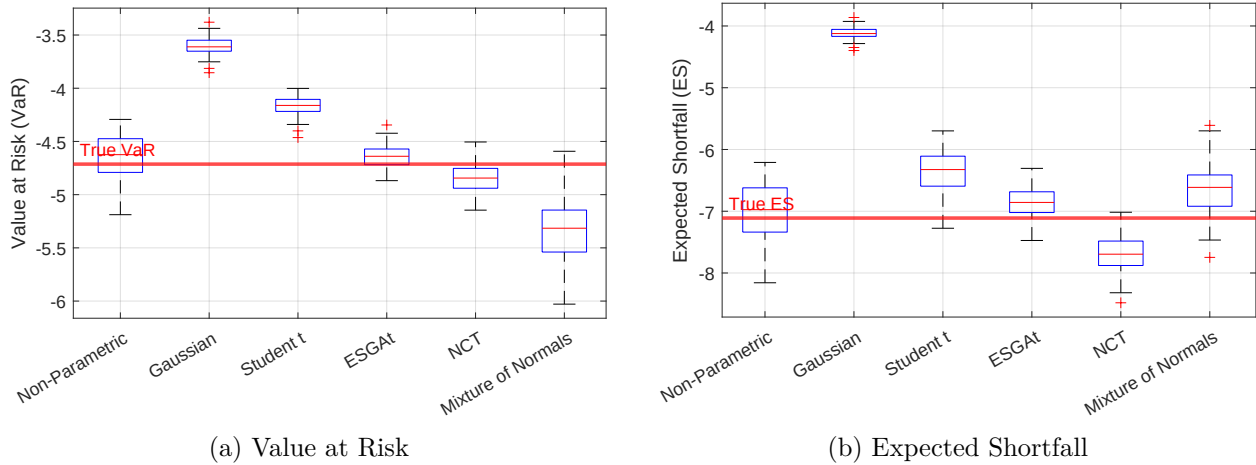


Figure 1: Boxplots of the VaR and ES, where GAt is DGP

Figure 2 shows two sample distributions, namely for location μ and skewness (θ) of our specified GAt-distribution. A similar pictures results for the other estimates hence they are omitted here.

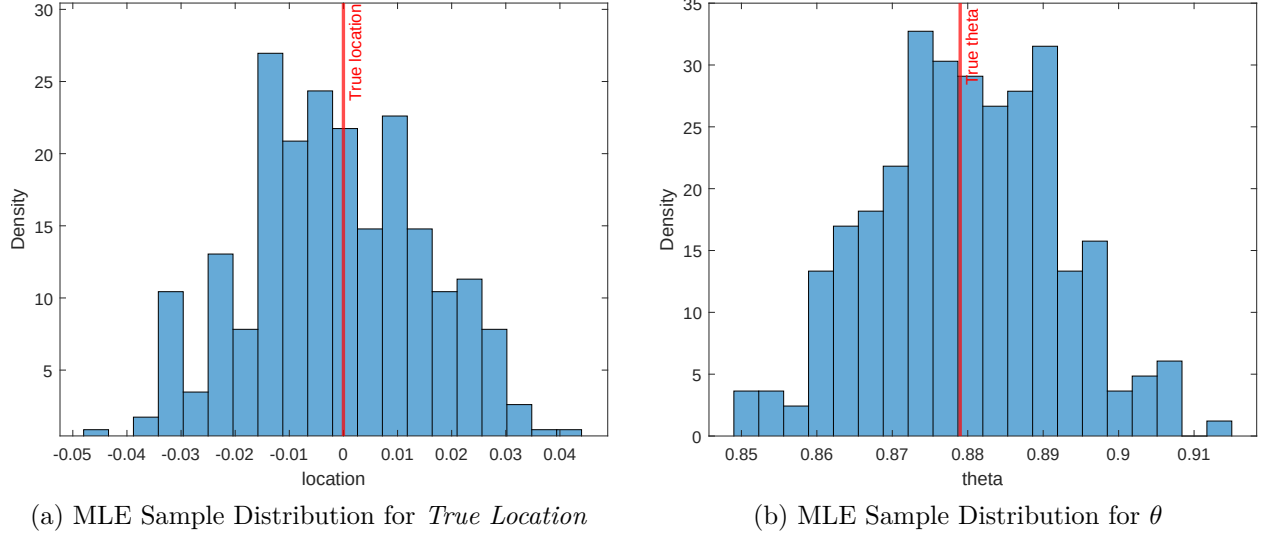


Figure 2: MLE for location zero GAt

To complete this part of the assignment we show Listing 4 which depicts how we obtain the bootstrap results. Each function shown here is given in the corresponding part of the Appendix. We use Listing 4 for all following exercises in Part I and only change the underlying data generating process. Each function inputs the data vector, a specified amount of resamples B where the amount of resamples in *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paoella, 2009) is $B = 100$. I guess this was done for computational efficiency.

Listing 4: MATLAB Code: Bootstrap

```

1 [VaR_studentt_values, ES_studentt_values] = compute_studentt_bootstrap(data, B, n,
  gamma);
2 [VaR_nonparametric_values, ES_nonparametric_values] =
  compute_nonparametric_bootstrap(data, B, n, gamma);
3 [VaR_mixture_values, ES_mixture_values] = mixture_of_normals(data, B, n, gamma);
4 [VaR_gaussian_values, ES_gaussian_values] = Gaussian_bootstrap(data, B, n, gamma);
5 [VaR_GA_t, ES_GA_t] = compute_parametric_bootstrap_GAt(data, B,n, gamma);
6 [VaR_ES_NCT, ES_NCT] = compute_parametric_bootstrap_NCT(data, B,n, gamma);

```

In Listing 5 we show how we computed the Expected Shortfall for the GAt Distribution essentially using (1). This is a piece of the code provided in Listing 29 in the appendix where we perform the full bootstrap assuming a GAt Distribution for our sample data.

Listing 5: MATLAB Code: True ES for GAt

```

1 ES_values(b) = mu_est+c_est*(((1-v_est^(1/d_est))*(1+theta_est^2))/((1 +
  theta_est^2)*theta_est))*(B_L_num/B_L_den);

```

This complete Exercise 1 of Part I for this assignment.

2. Expected Shortfall and Value at Risk Estimation with Student's t Distribution as Data Generating Process

This part essentially covers Take Home Assignment #1. We generate 10.000 observations from a Student's t-distribution with 4 degrees of freedom using Listing 6 and then proceed as before.

Listing 6: Generate a Random Data Set of IID Student's t Realizations in MATLAB

```
1 data = trnd(df, n, 1);
```

For each bootstrap sample, we estimate Value-at-Risk and Expected Shortfall. For the Nonparametric Method we basically compute the empirical quantile and empirical conditional mean. For the Gaussian Method we fit a normal distribution and compute the VaR and ES using the estimates for the parameters. The Student's t method well as anticipated and actually best in terms of variation around it's mean since the true data generating process is the Student's t itself. We fit a t-distribution and compute VaR and ES just as mentioned before. For the Mixture of Normals Method we fit a two-component Gaussian mixture distribution. The Gaussian method may mostly overestimates the risk measures, probably due to weighting of the components and generally due to lighter tails. The Nonparametric method shows higher variability but again performs quite well. All these results are in-line with the findings from *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paoletta, 2009) Figure 4.

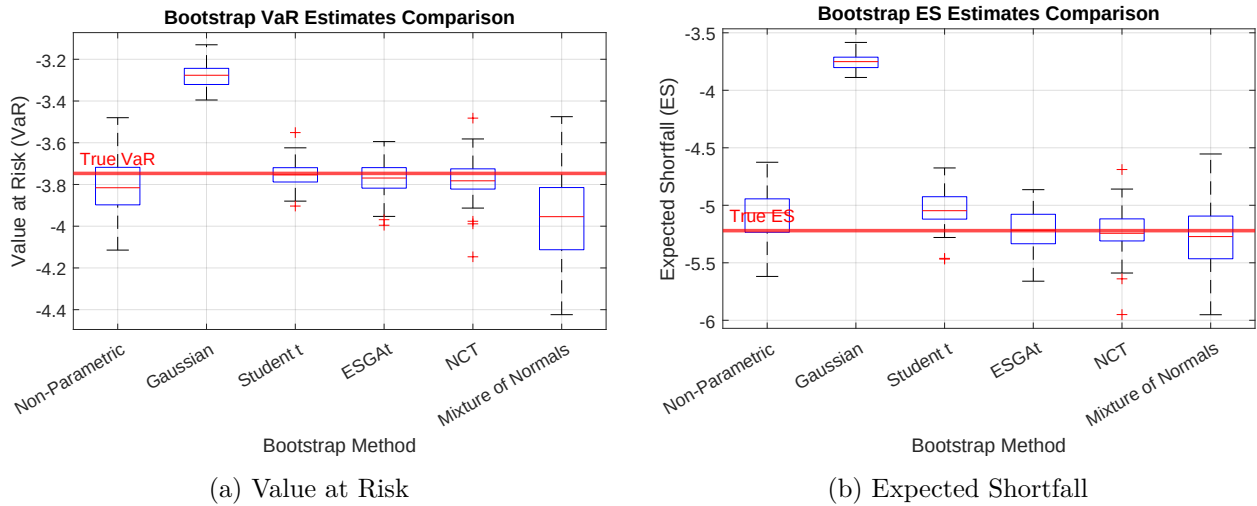


Figure 3: Boxplots of the VaR and ES, where Student's t is DGP

This complete the second part of the assignment. Estimation for the other methods (except student's t or GAt can be found in the appendix just in the previous exercise.

3. Expected Shortfall and Value at Risk Estimation with Asymmetric Stable Paretian Distribution as Data Generating Process

In this part of the assignment we take the Asymmetric stable Paretian distribution as our Data-generating process, so once again, we just change the DGP just as before. To obtain the asymmetric stable Paretian distributon one can extend the concept of the symetric stable Paretian distribution to allows for asymmetry as shown in *Intermediate Probability : A Computational Approach* (M. Paoletta, 2007). The distribution is stable, meaning any linear combination of independent random

variables with this distribution results in another random variable with the same distribution. Apart from that, its distribution also allows for skewness.

Please note that in all of the Figures the Stable Paretian corresponds to the "Pareto" description. I wrongly named them "Pareto" in MATLAB. Changing this part would require re-running the whole code which takes quite a lot of time for the Asymmetric Stable Paretian MLE (in Bonus part).

We approach our analysis in a similar way as in part 1. and 2.. Figure 6b was created using Listing 37 in the appendix. Here the true VaR, true ES and a part of the PDF relevant for our Expected Shortfall and Value-at-Risk analysis is shown.

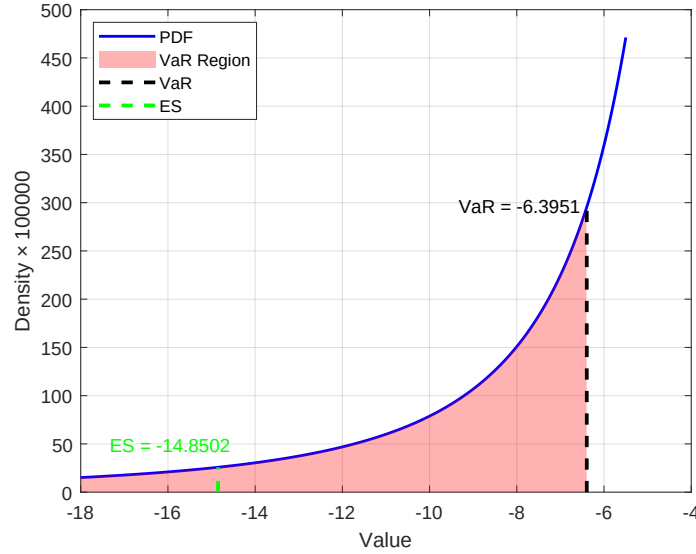


Figure 4: True Expected Shortfall for Asymmetric Stable Paretian, $\alpha = 1.7, \beta = -0.6$, zero location and unit scale

To simulate from the asymmetric stable Paretian we use Listing 7 based on the PDF and CDF of the asymmetric stable Paretian.

Listing 7: MATLAB Code for Simulating from Asymmetric Stable Paretian

```

1 function x = stabgen(nobs, a, b, c, d)
2     z = nobs;
3     V = unifrnd(-pi/2, pi/2, 1, z);
4     W = exprnd(1, 1, z);
5     if a == 1
6         x = (2/pi) * (((pi/2)+b*V).*tan(V)-b*log((W.*cos(V))./((pi/2)+b*V)));
7         x = c*x+d-(2/pi)*d*log(d)*c*b;
8     else
9         Cab = atan(b*tan(pi*a/2))/a;
10        Sab = (1+b^2*(tan((pi*a)/2))^2)^(1/(2*a));
11        A = (sin(a*(V+Cab))./((cos(V)).^(1/a)));
12        B0 = (cos(V-a*(V+Cab)))./W;
13        B = (abs(B0)).^((1-a)/a);
14        x = Sab*A.*(B.*sign(B0));
15        x = c*x+d;
16    end
17 end

```

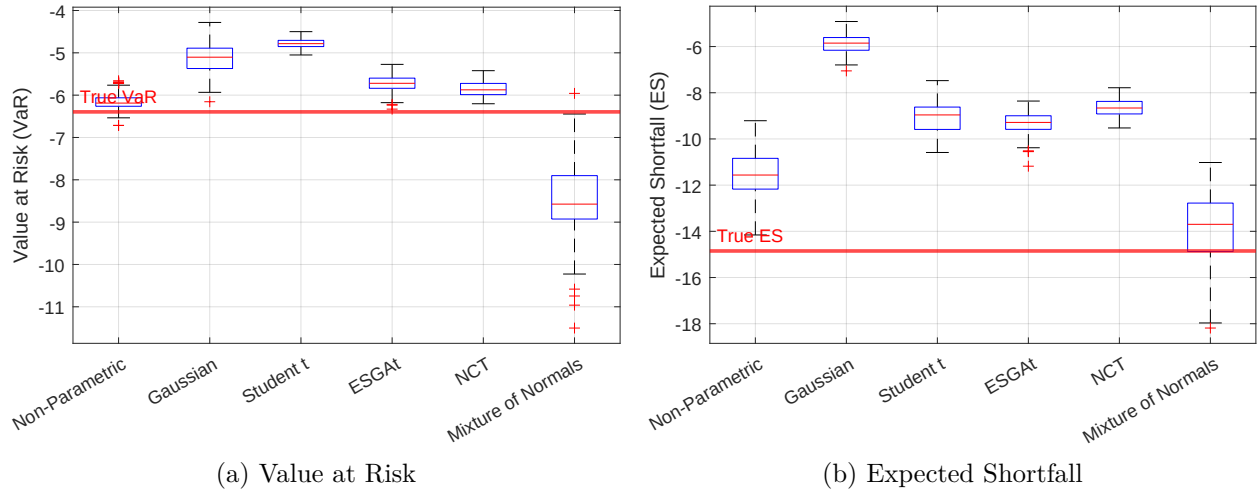


Figure 5: Boxplots of the VaR and ES, where Asymmetric Stable Paretian is DGP $N=10000$, $B=100$, $\alpha = 1.7$, $\beta = -0.6$, location zero and unit scale

Figure 5 shows the result when we change the data generating process to the asymmetric stable Paretian distribution, i.e. obtaining sample data from the aforementioned listing.

To compute the true Expected Shortfall and Value-at-Risk we use Listing 8. This function contains various helper functions all given in the appendix.

Listing 8: MATLAB Code for Computation of True ES for Asymmetric Stable Paretian

```

1 function [ES, VaR] = asymstableES(xi, a, b, mu, scale, method)
2     opt = optimset('Display', 'off', 'TolX', 1e-6);
3     q = fzero(@(x) stabcdfroot(x, xi, a, b), -6, opt);
4     VaR = mu+scale*q;
5     if q == 0
6         t0 = (1/a)*atan(b*tan(pi*a/2));
7         ES = ((2*gamma((a-1)/a))/(pi-2*t0))*(cos(t0)/cos(a*t0)^(1/a));
8         return;
9     end
10    if method == 1
11        ES = (scale*Stoy(q, a, b)/xi)+mu;
12    else
13        ES = (scale*stabetailcomp(q,a,b)/xi)+mu;
14    end
15 end

```

Figure 5 shows that the non-parametric bootstrap still performs reasonably well, at least for the VaR estimation. The results vary quite a lot with each new simulation but the overall picture remains the same.

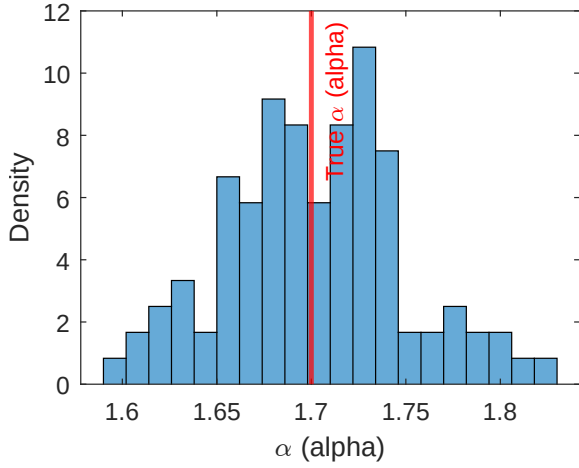
This completes the third part of the assignment.

3. Bonus: MLE for Asymmetric Stable Paretian

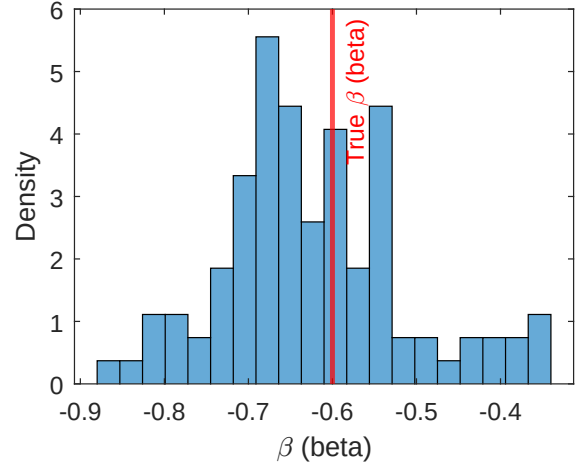
Let X be the asymmetric stable Paretian $X \sim S_{\alpha,\beta}(\mu, \sigma)$ with asymmetry parameter β for $-1 \leq \beta \leq 1$, its characteristic function, with $1 < \alpha \leq 2$, is given by

$$\phi_X(t; \alpha, \beta, \mu, \sigma) = \exp \left(i\mu t - |\sigma t|^\alpha \left[1 - i\beta \operatorname{sgn}(t) \tan \frac{\pi\alpha}{2} \right] \right), \quad 1 < \alpha \leq 2.$$

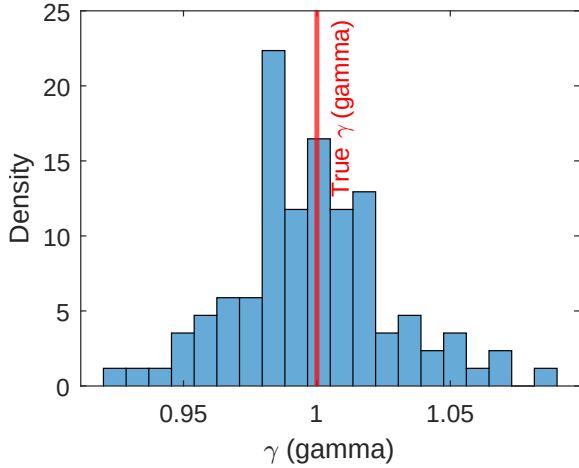
as shown in *Asymmetric stable Paretian distribution testing* (M. Paoletta, 2016). In this bonus part we compute the MLE for the asymmetric stable Paretian. Figure 6 shows histograms of the Stable's MLE results for $N = 250$ and $B = 100$ obtained from using the function for the MLE shown in Listing 38 in the appendix where the result shows asymptotic behavior regarding the true values.



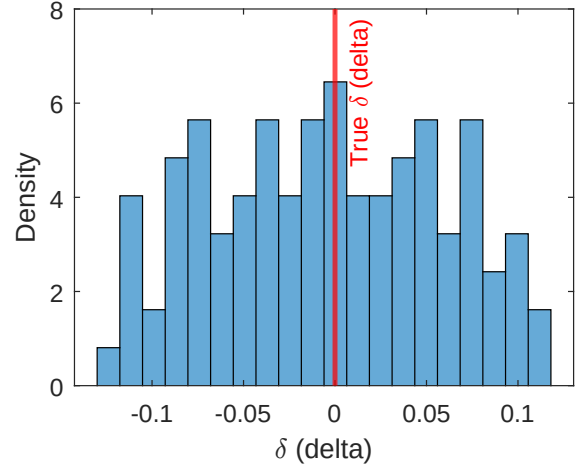
(a) Sample distribution of alpha



(b) Sample distribution of beta



(c) Sample distribution: Gamma here means scale parameter



(d) Sample distribution: Delta here means location parameter

Figure 6: MLE Sample Distribution for Asymmetric Stable Paretian given $N = 250$, $B = 100$, Duration: 21 Seconds for each b in $1:B$, $\alpha = 1.7$, $\beta = -0.6$, location zero and unit scale

The labeling here is not fully consistent from my side. By "delta" we mean the location parameter μ and by "gamma" we mean scale parameter σ .

Figure 7 replicates Figure 7 in *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paoletta, 2009), i.e. a small sample bootstrap estimation where the true DGP is the asymmetric stable Paretian distribution. In this small sample the estimation for the asymmetric stable Paretian performs best which is to be expected although due to the small there is quite a lot of variation.

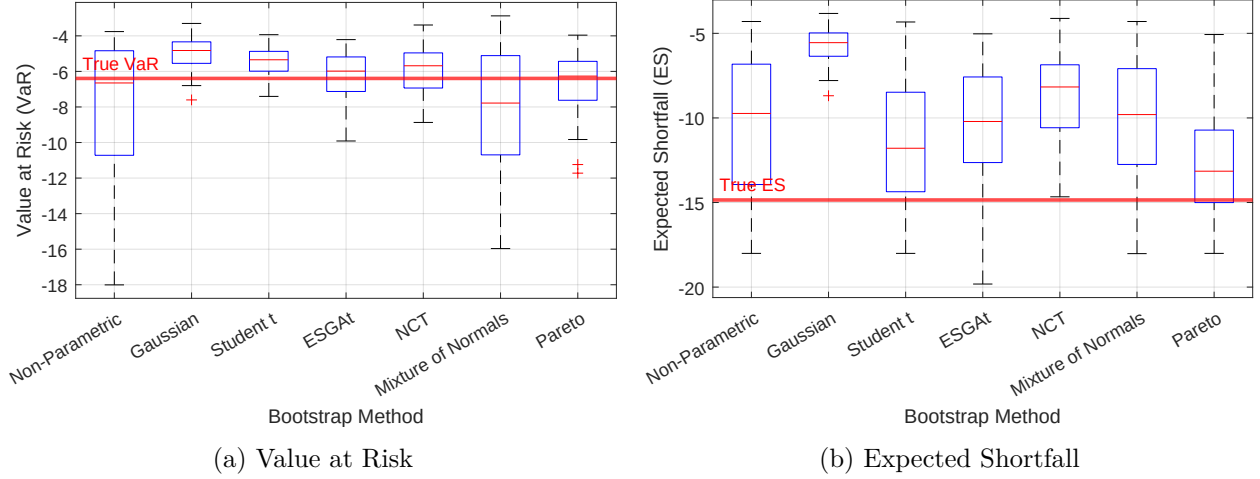


Figure 7: Reproduced Figure 7 *Calculating Expected Shortfall for Distributions in Finance* (S. Broda, M. Paoletta, 2009). Bootstrap using Asymmetric Stable Paretian as DGP, $N=250$, $B=200$ $\alpha = 1.7$, $\beta = -0.6$, location zero and unit scale

We then simply increase sample size to $N = 2500$ and change β to be -0.3 instead of -0.6 as before. Also here, the asymmetric stable paretian performs well and actually best in the Expected Shortfall case. Due to a smaller sample and changed parametrization this picture does not fully align in levels with Figure 8 but again, the overall picture remains the same.

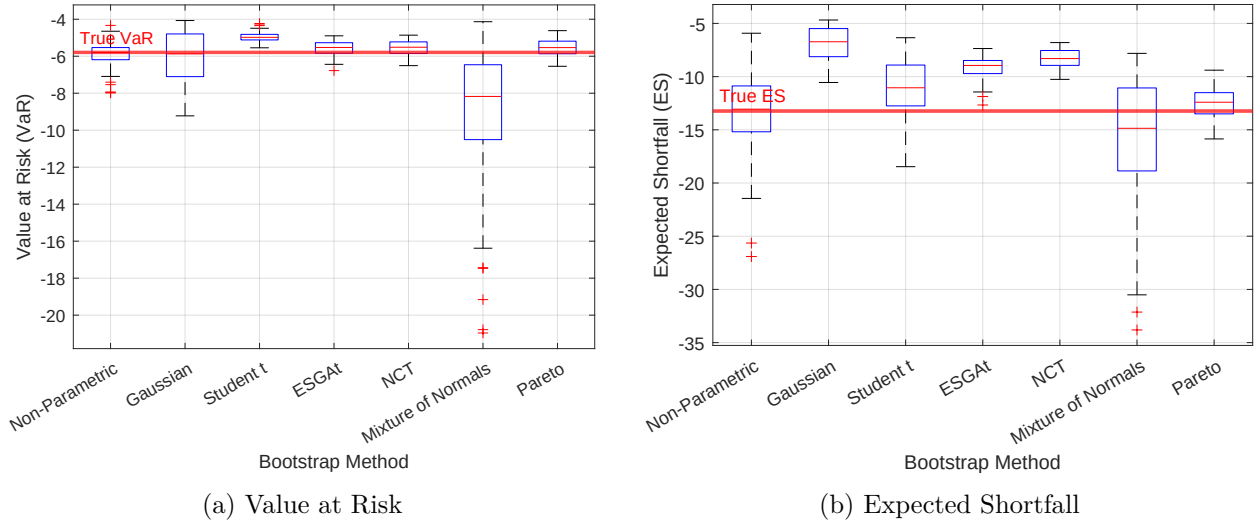


Figure 8: Same as in Figure 7, now $N=2500$, $B=100$, $\alpha = 1.7$, $\beta = -0.3$, location zero and unit scale

Note that we used $N = 10000$ and $\beta = -0.6$ in Figure 5, hence the VaR and ES estimates in this case (Bonus part) for the other distributions except the Paretian perform different although the overall pattern still looks much like what we have seen in Figure 5.

Part II

Traditional models, such as CCC or DCC GARCH, while foundational, fail to adequately capture the nuanced behavior of asset returns, necessitating more sophisticated distributional frameworks. This paper focuses exclusively on the distributional aspect of financial returns, deliberately setting aside time-varying dynamics (e.g., GARCH-type modeling) to explore static, IID-based non-elliptic distributions. Specifically, we introduce and implement methodologies for simulating and estimating two non-elliptic bivariate distributions: a discrete mixture of Laplace distributions and the multivariate noncentral Student t distribution (NCT).

A discrete mixture distribution is of particular relevance for financial returns data because of its ability to capture the following two additional stylized facts associated with multivariate asset returns: 1) The so-called leverage or down-market effect, or the negative correlation between volatility and asset returns. A popular explanation for this phenomenon is attributed to Black (1976), who noted that a falling stock price implies a higher leverage on the firm's capital structure (debt to equity ratio), and thus a higher probability of default. This increase in risk is then reflected in a higher stock price volatility.

1. Simulation of a Bivariate Discrete Mixture of Laplace Distributions for Financial Returns Modeling

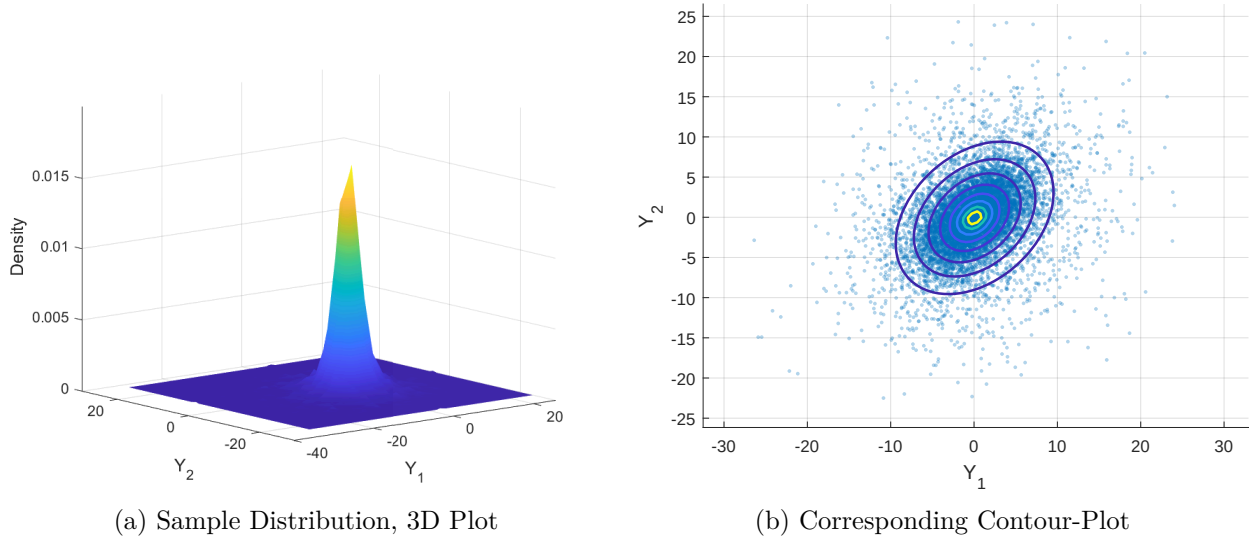


Figure 9: Samples from a Mixture of Multivariate Laplace Distributions, both Components are mean zero, Σ_2 has larger diagonal elements, level-colors align for both plots

The multivariate Laplace distribution can be represented as a normal variance-mean mixture. Specifically, if:

$$G \sim \text{Gamma}(b, 1),$$

where G is a Gamma-distributed random variable, and given G , $\mathbf{Y}$ follows a multivariate normal distribution:

$$\mathbf{Y} \mid G \sim \mathcal{N}_d(\boldsymbol{\mu}, G\boldsymbol{\Sigma}),$$

then the marginal distribution of $\mathbf{Y}$ (after integrating out G) follows a multivariate Laplace distribution with parameters $\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$, and b where d is the dimensionality. Hence:

$$f_Y(\mathbf{y}; \boldsymbol{\mu}, \boldsymbol{\Sigma}, b) = \int_0^\infty f_{Y|G}(\mathbf{y}; g) f_G(g) dg \quad (2)$$

as given in *Linear Models and Time-Series Analysis* (M. Paoletta, 2019).

We say a d -dimensional random variable follows a k -component mixture of multivariate Laplace distributions, or MixkLapd, if its p.d.f. is given by

$$f_{MixkLap_d}(\mathbf{y}; \mathbf{M}, \boldsymbol{\Psi}, \boldsymbol{\lambda}, \mathbf{b}) = \sum_{j=1}^k \lambda_j f_{Lap}(\mathbf{y}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j, b_j) \quad (3)$$

where $\lambda_j \in (0, 1)$, $\sum_{j=1}^k \lambda_j = 1$, with f_{Lap} denoting the d -variate multivariate Laplace distribution. The mixing proportions determine how likely a sample is to come from each component. This ensures the generated data respects the overall mixture proportions.

Listing 9: MATLAB Code for Comparison of Modified Bessel function of Third Kind

```

1 mu_list = {[ -2; -2], [2; 2]};
2 Sigma_list = {[1, 0.5; 0.5, 1], [8, -0.3; -0.3, 8]};
3 b_list = [2, 3];
4 lambda_true = [0.4, 0.6];
5 n = 10000;
6 [data, C] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda_true, n);

```

To simulate from a discrete mixture of two bivariate laplace distributions we use Listings 9 and 10 where we sample from the Gamma distribution $G_c \sim \text{Gamma}(b_c, 1)$ and separately sample from a multivariate normal distribution $\mathbf{Z}_c \sim (0, \boldsymbol{\Sigma}_c)$. We do that because we know that $\mathbf{Y} | G \sim \mathcal{N}_d(\boldsymbol{\mu}, G\boldsymbol{\Sigma})$. Why do we generally use the Laplace distribution? The Gaussian distribution could often be too "thin-tailed" to capture deviations or outliers while the Student's t-distribution might be too "heavy-tailed" for cases where extreme deviations are not as prominent.

By choosing the multivariate Laplace in a mixture, one component handles near-central values, while the second component can capture deviations or variability in the data.

Listing 10: MATLAB Code for Sample Data Simulation from *MixkLapd*

```

1 function [samples, C] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda, n)
2     C = zeros(1, n);
3     lambda_cum = cumsum(lambda);
4     U = rand(1, n);
5     C = discretize(U, [0; lambda_cum(:)]);
6     for c = 1:k
7         idx_c = find(C == c);
8         n_c = length(idx_c);
9         if n_c > 0
10             mu_c = mu_list{c};
11             Sigma_c = Sigma_list{c};
12             b_c = b_list(c);
13             G_c = gamrnd(b_c, 1, [1, n_c]);
14             Z_c = mvnrnd(zeros(1, d), Sigma_c, n_c)';
15             Y_c = mu_c + Z_c .* sqrt(G_c);
16             samples(:, idx_c) = Y_c;
17         end
18     end
19 end

```

The PDF for the multivariate laplace density is given in equation (4) and follows the derivation in *Linear Models and Time-Series Analysis* (M. Paoletta, 2019). We incorporate this density into our analysis by defining the function in Listing 11.

$$\int_0^\infty f_{Y|G}(\mathbf{y}; g) f_G(g) dg = \frac{1}{|\Sigma|^{1/2} (2\pi)^{d/2}} \frac{2}{\Gamma(b)} (m/2)^{b/2-d/4} K_{b-d/2}(\sqrt{2m}) \quad (4)$$

Listing 11: MATLAB Code Laplace Density (in levels and log)

```

1 function fY = laplace_density(y, mu, Sigma, b)
2     y = y(:);
3     mu = mu(:);
4     d = length(y);
5     diff = y-mu;
6     [L, p] = chol(Sigma, 'lower');
7     z = L\diff;
8     m = z' * z;
9     logDetSigma = 2*sum(log(diag(L)));
10    const1 = -0.5*(logDetSigma+d*log(2*pi));
11    const2 = log(2)-gamma1n(b);
12    exponent = (b/2)-(d/4);
13    nu = b-(d/2);
14    sqrt2m = sqrt(2*m);
15    if sqrt2m == 0
16        K_nu = Inf;
17    else
18        K_nu = bessellk(nu, sqrt2m);
19    end
20    logfY = const1+const2+exponent*log(m/2)+log(K_nu);
21    fY = exp(logfY);
22 end

```

Again, to obtain the density for the mixture of two bivariate laplace distributions which we will later need for the MLE in the negative log-likelihood function we use (3) which was implemented in where we essentially use the PDF of the bivariate Laplace distribution defined in 11. This completes the first exercise of Part II for this assignment.

2. Maximum Likelihood Estimation for Simulated Mixture Models and Parameter Comparison

In this part of the assignment we will first verify the Bessel function of third kind numerically and then proceed with the MLE for the mixture of two bivariate Laplace distributions.

2a. Verification of the Modified Bessel Function of the Third Kind through Numerical Integration

To validate the correctness of the modified Bessel function of the third kind, as given in equation 5, we propose a direct comparison between its numerical integral representation and the corresponding built-in function output *besselk* in MATLAB. This result is then compared to the output of the *besselk* function to ensure consistency and confirm its reliability.

$$K_v(x) = \frac{1}{2} \int_0^\infty u^{v-1} \exp \left[-x^2 \left(\frac{1}{u} + u \right) \right] du \quad (5)$$

Listing 12: MATLAB Code for Comparison of Modified Bessel function of Third Kind

```
1 for i = 1:length(v_values)
2     v = v_values(i);
3     for j = 1:length(x_values)
4         x = x_values(j);
5         integrand = @(u) u.^(v-1).*exp(-x/2*(u+1./u));
6         numerical_result = (1/2)*integral(@(u) integrand(u), 0, Inf, 'RelTol',1e
          -12, 'AbsTol',1e-12);
7         bessellk_result = bessellk(v, x);
8     end
9     error = abs(numerical_result-bessellk_result);
10 end
```

The output yields an absolute error between the result when using MATLAB's *bessellk* function compared to numerical integration of **1.1102e-16** i.e. a negligible difference. The code snippet below shows another possible method to calculate the integral using Trapezoidal rule or Simpson's rule. Since the upper limit is infinity, we truncate it to a large value. Then using Trapezoidal rule we approximate the integral and define a step-size for the approximation. Running this codes returns a result of **1.1102e-16** absolute error which is exactly the same as in the previous Listing.

Listing 13: MATLAB Code for Comparison using Trapezoidal Rule

```
1 v = 1;
2 x = 1;
3 integrand = @(u) (u.^(v-1)).*exp(-x*0.5*(u+1./u));
4 u_min = 1e-6;
5 u_max = 100;
6 n_points = 1e6;
7 u = linspace(u_min, u_max, n_points);
8 f_u = integrand(u);
9 delta_u = u(2)-u(1);
10 numerical_result = 0.5*delta_u*(f_u(1) + 2*sum(f_u(2:end-1))+f_u(end));
11 numerical_result = numerical_result*0.5;
12 matlab_result = bessellk(v, x);
13 error = abs(numerical_result-matlab_result);
```

Which completes this part of the assignment.

2b. Maximum Likelihood Estimation for a Bivariate Laplace Mixture Using BFGS Optimization

To compute the Maximum Likelihood Estimate (MLE) for a two-component, bivariate discrete mixture of Laplace distribution, we use the BFGS optimization algorithm to directly minimize the negative likelihood function. This approach contrasts Expectation-Maximization (EM) algorithm, bypassing the iterative expectation-maximization steps in favor of gradient-based optimization. We will use pre-existing code a different distribution given in *Linear Models and Time-Series Analysis* (M. Paoletta, 2019) and adapt this to the case of a mixture of two bivariate Laplace distributions.

The program will implement the negative log-likelihood function for the given Laplace mixture model, parameterized by the mixing proportions, location parameters, and scale parameters. The BFGS algorithm, with its efficient handling of gradient-based updates, will then minimize this function to estimate the parameters that maximize the likelihood of the observed data.

In this exercise we are making use of (2) and (5) by first defining Listing 14 to compute the pdf of the bivariate Laplace distribution.

Listing 14: MATLAB Code for PDF of Multivariate Laplace Distribution

```

1 function f = laplacepdf(x, mu, Sigma, b)
2     d = length(mu);
3     diff = x - mu;
4     [L, p] = chol(Sigma, 'lower');
5     if p > 0
6         f = realmin;
7         return;
8     end
9     z = L \ diff;
10    m = sum(z.^2);
11    log_det_Sigma = 2*sum(log(diag(L)));
12    log_const = log(2)-gammaln(max(b, realmin))-0.5*(log_det_Sigma+d*log(2*pi))+(b
        /2-d/4)*log(max(m/2,realmin));
13    nu = b-d/2;
14    sqrt2m = sqrt(2*m);
15    log_K = log(besselk(nu, sqrt2m) + realmin);
16    log_f = log_const + log_K;
17    f = exp(log_f);
18    if isnan(f) || isinf(f)
19        f = realmin;
20    end
21 end

```

We then continue by defining the negative log likelihood function in order to use the BFGS algorithm implemented in *fminunc* in MATLAB. This function tries to obtain the minimum, hence we define the *negative* log likelihood function *negloglik*.

We use the function defined in 15 which is the PDF of a bivariate Laplace distribution as well as (3) to obtain the mixed bivariate Laplace distribution assuming values for λ , for our simulations following specifically $\lambda_1 = 0.4, \lambda_2 = 0.5$, which are our mixing probabilities for the two independent bivariate Laplace distributions.

Listing 15: MATLAB Code for the Likelihood Function of the Multivariate Laplace

```

1 function negloglik = negloglik_laplace(param, data, k)
2     [d, n] = size(data);
3     gamma = param(1:(k-1));
4     lambda = [exp(gamma)./(1+sum(exp(gamma))), 1/(1+sum(exp(gamma)))];
5     mu = reshape(param(k:(k+k*d-1)), [d, k]);
6     cov_params = param((k+k*d):(k+k*d+k*3-1));
7     b = exp(param((k+k*d+k*3):end));
8     Sigma_list = cell(k, 1);
9     idx = 1;
10    for i = 1:k
11        L_vec = cov_params(idx:(idx + 2));
12        idx = idx+3;
13        L = [exp(L_vec(1)), 0; L_vec(2), exp(L_vec(3))];
14        Sigma_list{i} = L*L';
15    end
16    ll = 0;
17    for i = 1:n
18        xi = data(:, i);
19        density_i = 0;
20        for j = 1:k
21            density_ij = lambda(j)*laplacepdf(xi, mu(:, j), Sigma_list{j}, b(j));
22            density_i = density_i+density_ij;
23        end
24        ll = ll + log(density_i);
25    end
26    negloglik = -ll;
27 end

```

We then compute the density of the discrete mixture of two bivariate Laplace distributions in line 21 in Listing 15 weighting each of the k components (in our case $k = 2$) by λ_j .

Listing 16 shows the application of the previous results, i.e. the MLE for a mixture of k bivariate Laplace distributions. Clearly, we need to define initial values for which I tried to come up with an optimization that picks somewhat optimal values based on the sample data and is shown in Listing 41 in the Appendix.

Listing 16: MATLAB Code for MLE of Mixture of Bivariate Laplace Distributions

```

1 param_init = [gamma_init, mu_init(:)', cov_params_init, b_init];
2 options = optimoptions('fminunc', 'Algorithm', 'quasi-newton', 'MaxIterations',
3     1000);
4 negloglik_est = @(param) negloglik_laplace(param, data, k);
5 [param_est, fval] = fminunc(negloglik_est, param_init, options);

```

To give an illustration of the MLE for the mixture of two bivariate Laplace distributions, we define $\mu_1 = 2, \mu_2 = 2, \sigma_{11} = 1, \sigma_{12} = 0.5, \sigma_{21} = 0.5, \sigma_{22} = 1$, for the first mixture component and take values: $\mu_1 = -6, \mu_2 = -4, \sigma_{11} = 5, \sigma_{12} = 3, \sigma_{21} = 3, \sigma_{22} = 5$ for the second component which is summarized in Table 1. The chosen parameters λ are $\lambda_1 = 0.4, \lambda_2 = 0.6$.

In order to obtain the standard errors we first define the hessian provided by *fminunc*, then define the variance-covariance matrix of the estimates and finally extract the standard errors from the diagonal of the variance-covariance matrix which is shown in Listing 17. The following tables show our results.

Parameter	μ_1	μ_2	σ_{11}	σ_{12}	σ_{21}	σ_{22}	b
Laplace 1							
True Value	2	2	1	0.5	0.5	1	10
Estimate	2.2378 (0.1132)	2.1231 (0.0984)	0.9804 (0.1874)	— —	0.5154 (0.1005)	1.0738 (0.1870)	8.7632 (0.3435)
Laplace 2							
True Value	-6	-4	5	3	3	5	5
Estimate	-5.7290 (0.1932)	-3.7345 (0.1522)	5.3311 (0.0588)	— —	3.3967 (0.0903)	5.3279 (0.0647)	4.9558 (0.1205)

Table 1: True Parameters and Estimates, Standard Errors are in Paranthesis and $N = 10000$

Listing 17: MATLAB Code for Standard Errors of Estimates resulting from MLE

```

1 [~, ~, ~, ~, ~, hessian] = fminunc(negloglik_fun, param_est, options);
2 varcov_param = inv(hessian);
3 se_param = sqrt(diag(varcov_param));

```

To give another illustration, in the following example, Table 2 shows the results from using mean zero vectors for our simulation study of the MLE with again mixing probabilities of $\lambda_1 = 0.4$, $\lambda_2 = 0.5$, so we put more weight on the volatile component. The corresponding sample plot is shown in Figure 10 where the contours are based on the PDF evaluated at our estimators from Table 2. The *red X* indicates the estimated means, i.e. both close to true value 0.

Parameter	μ_1	μ_2	σ_{11}	σ_{12}	σ_{21}	σ_{22}	b
Laplace 1							
True Value	0	0	1	0.5	0.5	1	10
Estimate	0.0691 (0.1068)	0.0180 (0.1090)	1.0934 (0.1354)	— —	0.5672 (0.0970)	1.1095 (0.1590)	9.8888 (0.5332)
Laplace 2							
True Value	0	0	5	4.5	4.5	5	5
Estimate	0.0562 (0.1190)	0.0432 (0.0572)	4.7945 (0.0988)	— —	4.3328 (0.1003)	4.7844 (0.0737)	5.8282 (0.1102)

Table 2: True Parameters and Estimates, Standard Errors are in Paranthesis and $N = 10000$

Figure 10 completes this part of the assignment where we show the corresponding scatter plot including the contours obtained from evaluating the PDF (vectorized) at the estimates from Table 2.

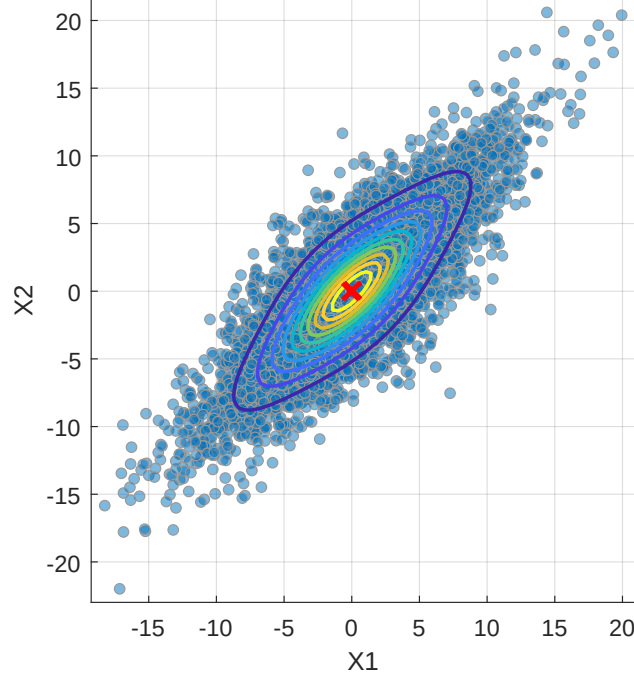


Figure 10: Fitted Contours, based on MLE from Table 2, $N=10000$

3. MLE Computation for Bivariate NCT and Mixture of Bivariate Laplace Distributions

In this part of the exercise, we will create a program to compute the Maximum Likelihood Estimator of a bivariate NCT distribution given a data set of $T \times 2$. The program outputs the MLE parameter vector. This second program integrates the functionality of the first two and additionally calculates and outputs the AIC and BIC values for each model. These metrics facilitate in-sample model comparison, allowing for the determination of the model that best fits the data. For AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion), the goal is to select the model with smaller values of AIC and BIC because we are penalizing the model specifications.

Table 5 shows results obtained from the MLE for the Multivariate NCT (MVNCT). The data was simulated from the MVNCT as the true data generating process briefly verifying our MLE function as shown in Listing 18 which shows the MLE code for the Multivariate Noncentral Student's-t as given by *Linear Models and Time-Series Analysis* (M. Paoletta, 2019).

Parameter	Estimated	True
Degrees of Freedom (v)	5.2105	5.0000
Non-centrality Parameters (μ)	-0.0109, -0.0024	0.0000, 0.0000
Scale Parameters	1.0619, 0.9599	1.1000, 0.9000
Correlation	0.3126	0.3000

Table 3: MLE for Bivariate NCT: Comparison of Estimated and True Parameters, $N = 10000$

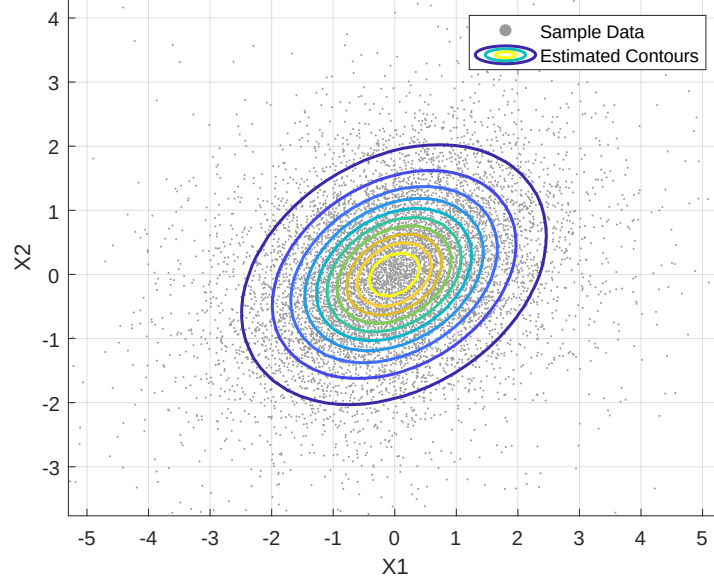


Figure 11: Corresponding Contourplot to Table 5, DGP is NCT, Contours based on MLE NCT, $N=10000$

Listing 18: MLE of MVNCT, *Linear Models and Time-Series Analysis* (M. Paoletta, 2009)

```

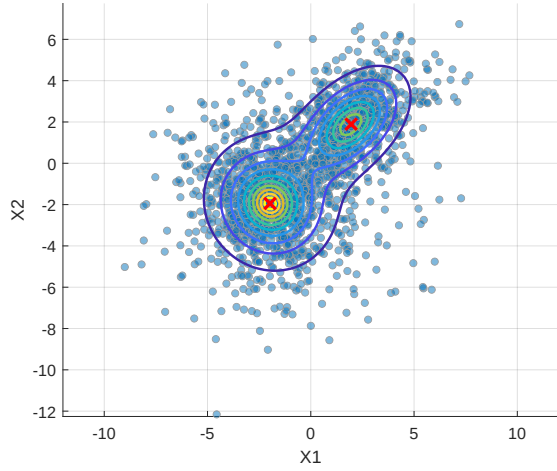
1 function [param, stderr, loglik, Varcov] = MVNCT2estimation(data)
2     [d, T] = size(data);
3     bound.lo = [1.1, -1, -1, 0.01, 0.01, -1, -4, -4];
4     bound.hi = [20, 1, 1, 100, 100, 1, 4, 4];
5     bound.which = [1, 0, 0, 1, 1, 1, 1, 1];
6     initvec = einschraenk(initvec, bound);
7     maxiter = 300;
8     tol = 1e-6;
9     opts = optimset('MaxIter', maxiter, 'TolFun', tol, 'TolX', tol, 'LargeScale',
10                     'off');
11     obj_fun = @(param) MVNCTloglik(param, data, bound);
12     [pout, fval, ~, output, ~, hess] = fminunc(obj_fun, initvec, opts);
13     [param, Varcov] = einschraenk(pout, bound, inv(hess) / T);
14     stderr = sqrt(diag(Varcov));
15     loglik = -fval * T;
16 end

```

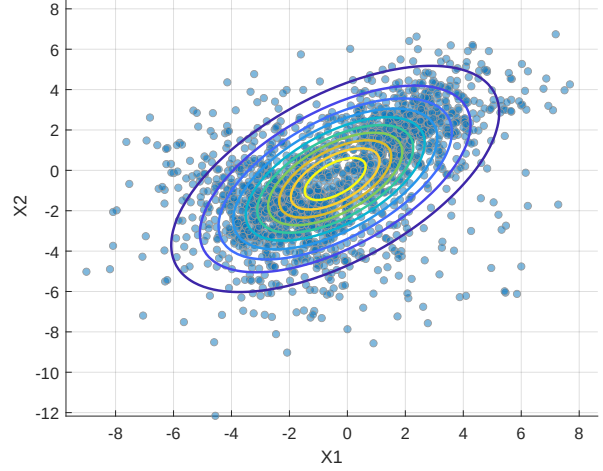
We then continue by simulating 2500 observations from a mixture of two bivariate Laplace distributions as before in part 2.b, but now also compute the MLE for the NCT. Table 4 shows the AIC, BIC and Log-Likelihood based on a single sample distribution. Clearly, the estimation for the mixture of bivariate laplace works better when the underlying data is actually the corresponding mixture of bivariate Laplace distributions, i.e. clearly showing lower AIC and BIC values.

Model	Log-Likelihood	AIC	BIC
Laplace Mixture	-6491.88	13009.75	13085.47
MVNCT	-13705.57	27427.15	27473.74

Table 4: Model Comparison when DGP is Mixture of two Bivariate Laplace, $N=2000$



(a) Simulated Data, MLE for Mixture of Laplace



(b) Scatter same as in Figure 15a MLE for MVNCT

Figure 12: Plots Corresponding to Table 8, true DGP is Laplace Mixture, Scatter Plot is the same both Graphs, True DGP has: $\mu_1 = (-2, -2)'$, $\mu_2 = (2, 2)'$, $b = (3, 2)'$, Σ_2 "large"

Figure 15 shows the corresponding contour plots. Both have the same underlying data points, although the axis are a bit skewed by MATLAB. We can see that fitting the Mixture of Laplace allows for more flexibility than the simple bivariate NCT which does not capture the two differing mean vector components that we used to simulated the underlying data. For all previous graphs the contours were computed using a vectorized form of the laplace density (or MVNCT respectively). The vectorized form is given in Listing 19. I used some "tricks" to make sure the resulting PDF levels are stable using $\log(\text{besselk}(\text{nu}, \text{sqr}t{2m} + \text{realmin}) + \text{realmin})$ for instance which allows to control for instability resulting from low levels at which the logarithm is evaluated.

Listing 19: Vectorized Laplace Density for Contour Plot

```

1 function fY = laplace_density_vectorized(Y, mu, Sigma, b)
2     mu = mu(:);
3     d = length(mu);
4     N = size(Y, 2);
5     diff = Y - mu;
6     [L, p] = chol(Sigma, 'lower');
7     Z = L \ diff;
8     m = sum(Z.^2, 1);
9     logDetSigma = 2*sum(log(diag(L)));
10    const1 = -0.5*(logDetSigma+d*log(2*pi));
11    const2 = log(2)-gammaln(b);
12    exponent = (b/2)-(d/4);
13    nu = b - (d/2);
14    sqrt2m = sqrt(2*m);
15    K_nu = log(besselk(nu, sqrt2m+realmin)+realmin);
16    K_nu(sqrt2m == 0) = Inf;
17    logfY = const1+const2+exponent*log(max(m/2,realmin)) + K_nu;
18    fY = exp(logfY);
19    fY(~isfinite(fY)) = 0;
20 end

```

Simulating data from a multivariate NCT for one large element of the diagonal elements in the variance-covariance matrix does not harm estimate accuracy at all but increases AIC and BIC greatly: $\mu = (0, 0)'$, $\nu = 4$, $\sigma_{11} = 20$, $\sigma_{22} = 1$, $R = 0.5$. Results are shown in Figure 13.

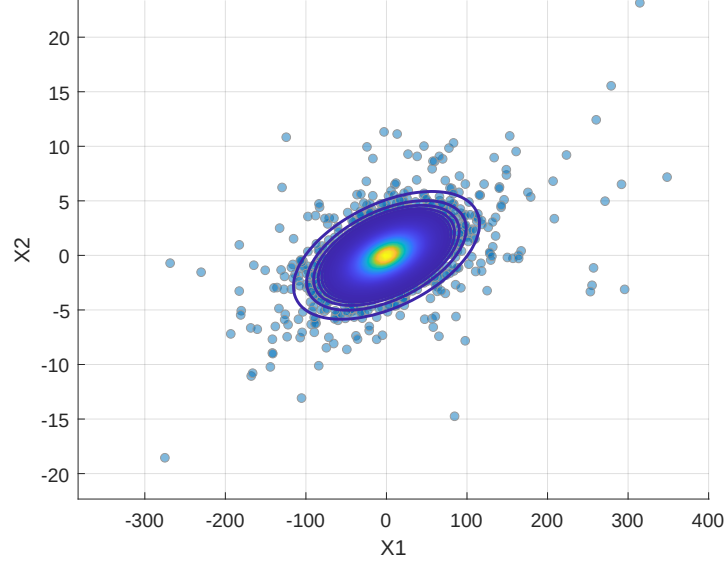


Figure 13: Contours are the PDF evaluated at the MLE, Large Variance for one Element and $N = 10000$

Parameter	Estimated	True
Degrees of Freedom (v)	3.0097	3.0000
Non-centrality Parameters (μ)	-0.0416, -0.0266	0.0000, 0.0000
Scale Parameters	20.0590, 1.0171	20.0000, 1.0000
Correlation	0.4902	0.5000
Log-Likelihood	-51045.6733	—

Table 5: MLE for Bivariate NCT: Results corresponding to Figure 13.

We now repeating the analysis in Table 4 multiple times using $N=100$ (for fast evaluation) and 100 repetitions and where the true DGP is a mixture of two bivariate Laplace distributions. Using Listing 20 we compute the average amount of AICs being smaller for MixLap than MVNCT.

Listing 20: Model preferred by AIC where True DGP is Mixture of Laplace in a Small Sample

```
1 mean(abs(AIC_MixLap)<abs(AIC_NCT))
```

We do the same for BIC and obtain the following table. So clearly the penalty term of BIC becomes relatively more impactful compared to AIC's penalty term in a small data set. The 5 additional parameters in the Mixed Laplace model are more heavily penalized by BIC than by AIC, especially with our small sample size.

Metric:	AIC	BIC
In favor of MixLap	86.00%	52.00%

Table 6: Favored Model by AIC or BIC for (only!) $N=100$, 100 Resamples, True DGP is Mixture of Laplace Distributions

Below we see the exactly same except now the DGP is MVNCT. I have to note that there is quite some variation. Running this a few times obviously lets the "favor-probability" jump a lot where sometimes the MixLap is still preferred in 60% of the cases. However, overall the probability of favoring the MixLap is lower on average if the true DGP is MVNCT which is what is to be expected. In every new approach, the BIC percentage is lower than the AIC in this small sample setting.

Metric:	AIC	BIC
In favor of MixLap	20.00%	18.00%

Table 7: Favored Model by AIC or BIC, N=100, 100 Resamples, True DGP is MVNCT

4. Model Comparison of Bivariate Mixture-Laplace and NCT Distributions on DJIA Stock Pairs: Empirical Application

For this part of the assignment we will specifically use Listing 44 which is not provided here due to its length. All functions are interactive with each other in some way so the important parts are summarized here. We can use our previous results, specifically Listing 16 and Listing 18 where we estimate first the MLE for a mixture of two bivariate Laplace distributions and second the bivariate NCT. We can simply change the data origin, i.e. stop using our simulated data. All of this was completely done without outlier handling. Taking out some outliers would first require an algorithm to detect those but I expect this to decrease the AIC and BIC, i.e. a smaller negative log likelihood (in absolute terms) which I neglect for this exercise. Hence the AIC and BIC values might appear quite large in value terms, although interpreting the level of AIC and BIC may not be appropriate anyways.

In Listing 21 we comment out the simulated data, i.e. ignore sample data from a mixture of two bivariate Laplace distributions and instead use the function shown in Listing 22 which selects two pairs in the data by using a correlation-based metric to guide the relevance of pairs.

Listing 21: MATLAB Code for using DJIA Data

```

1 [%[data, C] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda_true, n);
2 data = DJIA_stock_selection();

```

Highly correlated stocks are more likely to be selected, as they may exhibit more meaningful joint behavior. We normalized the absolute values of the correlation matrix to create a probability distribution, i.e. line 17 ensures that we are using a (by definition) correct probability measure. Pair selection was performed using the *randsample* function. The upper triangular portion of the correlation matrix was used to avoid duplicate pairs and self-pairs. This ensures that the selection process is both randomized and guided by a metric.

Moreover, my thought was that it could be interesting to consider the correlation between the selected stocks since you are interested in fitting a bivariate distribution, selecting stocks that exhibit varying degrees of correlation could provide more some interesting results.

Listing 22: MATLAB Code for Selecting Pairs in DJIA Data

```

1 function data_djia = DJIA_stock_selection()
2     fileName = 'DJIA30stockreturns.mat';
3     data = load(fileName);
4     data = data.DJIARet;
5     numStocks = size(data, 2);
6     n = size(data, 1);
7     pairs_selected = zeros(num_observations, 2);
8     corrMatrix = corr(returns);
9     corrMatrix(logical(eye(size(corrMatrix)))) = -Inf;
10    for rep = 1:numPairs
11        [row, col] = find(triu(corrMatrix, 1));
12        probs = abs(corrMatrix(sub2ind(size(corrMatrix), row, col)));
13        probs = probs/sum(probs);
14        selectedIdx = randsample(length(probs), 1, true, probs);
15        pair = [row(selectedIdx), col(selectedIdx)];
16        pairData = data(:, pair);
17        dataX = pairData;
18        dataX = dataX.';
19    end
20    data_djia = dataX;
21 end

```

Again, in order to compute the MLE for the mixture of two bivariate Laplace distributions we use 23 where we weight two individual bivariate Laplace distributions by λ looped over the $k = 2$ components.

Listing 23: MATLAB Code for Computation of Mixed Laplace Density

```

1 for j = 1:k
2     log_density_j = laplacepdfln(xi, mu(:,j), Sigma_list{j}, b(j));
3     mix_density = mix_density+lambda(j)*exp(log_density_j);
4 end

```

One option could be to standardize the data (which is done for the multivariate NCT). After testing some variation in our code I found that standardizing does not really help in getting more accurate estimators (in simulated data) so I stick to non-standardized data for the Laplace mixture. Once again, starting values for the MLE of the Laplace mixture result from Listing 41 which I found to be quite good in the simulation setting.

Listing 24: MATLAB Code for AIC and BIC Calculation

```

1 num_params_laplace = length(param);
2 aic_laplace = 2*num_params_laplace-2*loglik;
3 bic_laplace = num_params_laplace*log(n)-2*loglik;

```

In Listing 24 we show how we calculate the AIC and BIC values, here for the Laplace mixture using the following two equations based on the definition of the AIC and BIC

$$\text{AIC} = 2k - 2 \ln(\hat{L})$$

$$\text{BIC} = k \ln(n) - 2 \ln(\hat{L})$$

Also in the code provided in the appendix one could try it out and switch between the underlying data origin (i.e. DGE Mixture of Laplace, DGE MVNCT or DGE DJIA data) by commenting out or not in Listing 25 which is fully provided in Listing 44.

Listing 25: MATLAB Code for Selecting Pairs in DJIA Data

```
1 [data, C] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda_true, n);
2 %data = DJIA_stock_selection();
3 %data = MVNCTsim();
```

Model	Average AIC	Average BIC
Laplace Mixture	59558	59649
MVNCT	59652	59707

Table 8: Model Comparison for full Sample, 8000 Observations, 50 Stock Pick Average

Based on AIC the Mix_dLap_k Model is preferred in 92% of the cases which is only true for 84% of the cases using BIC (for this specific 50 selections we ran) In the Figure above we show some AIC and BIC values. The are overall very close although AIC and BIC for Laplace are mostly slightly greater. Now this result can change quite a lot depending on the Bessel function we use.

But I have to note that the AIC and BIC values were very close to each and also sometimes my code crashed and the MLE did not converge due to some numerical issues with the Bessel function.

Metric:	AIC	BIC
In favor of MixLap	92.00%	84.00%

Table 9: Favored Model by Metric AIC or BIC, DJIA Data

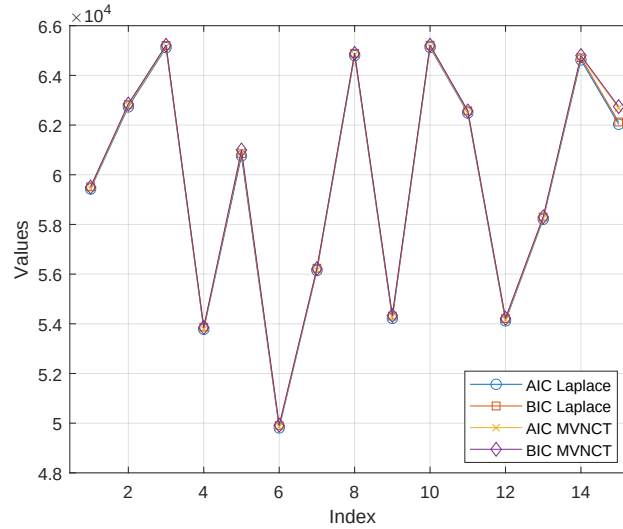
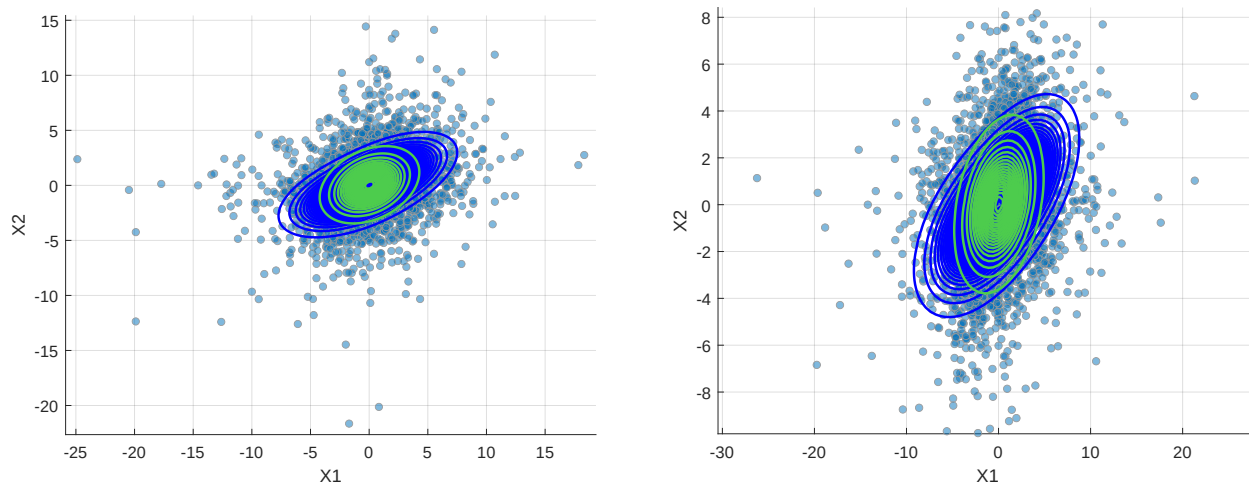


Figure 14: Plot for some AIC and BIC for various Stock Pairs (x-axis: Different Pairs), not a Time Series

The following figure completes the assignment and shows the estimated contours (PDFs evaluated at MLE) for two stock picks. The green contour is based on the MVNCT, the blue is based on the mixture of two bivariate Laplace distributions. I also found that switching the code for the Bessel function, i.e. using the suggestion in *Linear Models and Time-Series Analysis* (M. Paolella, 2019) on p.650 always gives me **100%** preference for the Mix. Laplace model. This alternative code is given

in Appendix Listing 45 and is extremely fast, i.e. almost instantaneous. However, I have noticed that in the simulation setting, this code performs not as good as when using the other approach.



(a) Simulated Data, MLE for Mixture of Laplace

(b) Scatter same as in Figure 15a MLE for MVNCT

Figure 15: Two stock picks and their estimated contours (PDF evaluated at MLE)

The table below shows the alternative approach from Appendix Listing 45 using the suggestion by *Linear Models and Time-Series Analysis* (M. Paoella, 2019) on p.650 when choosing **100** instead of **50** stock pairs.

Metric:	AIC	BIC
In favor of MixLap	99.00%	99.00%

Table 10: Favored Model by Metric AIC or BIC, DJIA Data, 100 Pairs

This completes the assignment.

Appendices

MATLAB Code for Part I Task 1: GAt Quantile Function

Listing 26: MATLAB Code for GAt quantile function

```
1 function q = GAtquantile(p, d, v, theta)
2     lobound = -100;
3     hibound = 100;
4     while true
5         [~, cdf_lobound] = GAt(lobound, d, v, theta);
6         f_lobound = cdf_lobound - p;
7         if f_lobound <= 0
8             break;
9         else
10            lobound = lobound - 10;
11            if lobound < -1e6
12                end
13            end
14        end
15        while true
16            [~, cdf_hibound] = GAt(hibound, d, v, theta);
17            f_hibound = cdf_hibound - p;
18            if f_hibound >= 0
19                break;
20            else
21                hibound = hibound + 10;
22                if hibound > 1e6
23                    end
24                end
25            end
26        tol = 1e-8;
27        opt = optimset('display', 'off', 'TolX', tol);
28        q = fzero(@(x) ff(x, p, d, v, theta), [lobound, hibound], opt);
29
30        function out = ff(x, p, d, v, theta)
31            [~, cdf_val] = GAt(x, d, v, theta);
32            out = cdf_val - p;
33        end
34    end
35    function [pdf, cdf, themean, thevar, quant, theES] = GAt(xvec, d, v, theta,
36        gammaforES)
37        ll = length(xvec);
38        xvec = reshape(xvec, ll, 1);
39        pdf = zeros(ll, 1);
40        konst = 1 / (beta(1/d, v) * v^(1/d) * (theta + 1/theta) / d);
41        k = find(xvec < 0);
42        if any(k)
43            y = xvec(k);
44            pdf(k) = (1 + (-y * theta).^d / v).^(-(v + 1/d));
45        end
46        k = find(xvec >= 0);
47        if any(k)
48            y = xvec(k);
49            pdf(k) = (1 + (y / theta).^d / v).^(-(v + 1/d));
50        end
51        pdf = konst * pdf;
```

```

51     if nargout > 1
52         cdf = zeros(length(xvec), 1);
53         k = find(xvec < 0);
54         if any(k)
55             y = xvec(k);
56             L = v ./ (v + (-y * theta).^d);
57             cdf(k) = betainc(L, v, 1/d) / (1 + theta^2);
58         end
59         k = find(xvec == 0);
60         if any(k)
61             cdf(k) = 1 / (1 + theta^2);
62         end
63         k = find(xvec > 0);
64         if any(k)
65             y = xvec(k);
66             top = (y / theta).^d;
67             U = top ./ (v + top);
68             cdf(k) = 1 / (1 + theta^2) + betainc(U, 1/d, v) / (1 + theta^(-2));
69         end
70     end
71 end

```

MATLAB Code for Part I Task 1: ES and VaR for all Distributions

Listing 27: MATLAB Code for Bootstrap assuming Student's t Distributon

```

1 function [VaR_values, ES_values] = compute_studentt_bootstrap(data, B, n, gamma)
2     VaRlevel = gamma;
3     ESlevel = gamma;
4     VaR_values = zeros(B, 1);
5     ES_values = zeros(B, 1);
6     for b = 1:B
7         ind = unidrnd(n, [n, 1]);
8         bootsamp = data(ind);
9         VaR_bootstrap_np = quantile(bootsamp, VaRlevel);
10        VaR_values(b) = VaR_bootstrap_np;
11        ES_values(b) = mean(bootsamp(bootsamp <= VaR_bootstrap_np));
12    end
13 end

```

Listing 28: MATLAB Code for Bootstrap assuming Gaussian Distribution

```

1 function [VaR_values, ES_values] = Gaussian_bootstrap(data, B, n, gamma)
2     ESlevel = gamma;
3     VaR_values = zeros(B, 1);
4     ES_values = zeros(B, 1);
5     for b = 1:B
6         ind = unidrnd(n, [n, 1]);
7         bootsamp = data(ind);
8         [mu_hat, sigma_hat] = normfit(bootsamp);
9         VaR_normal = norminv(ESlevel, mu_hat, sigma_hat);
10        VaR_values(b) = VaR_normal;
11        ES_normal = mu_hat - sigma_hat * normpdf(norminv(gamma)) / gamma;
12        ES_values(b) = ES_normal;
13    end
14 end

```

Listing 29: MATLAB Code for Bootstrap assuming GAt Distribution

```

1 function [VaR_values, ES_values] = compute_parametric_bootstrap_GAt(data, B,n,
2   gamma)
3   ESlevel = gamma;
4   VaR_values = zeros(B, 1);
5   ES_values = zeros(B, 1);
6   for b = 1:B
7       ind = unidrnd(n, [n, 1]);
8       bootsamp = data(ind);
9       [param_est, ~, ~, ~, ~] = GAtestimation(bootsamp, 0.01, [], []);
10      d_est = param_est(1);
11      v_est = param_est(2);
12      theta_est = param_est(3);
13      mu_est = param_est(4);
14      c_est = param_est(5);
15      VaR_values(b) = mu_est + c_est * GAtquantile(gamma, d_est, v_est,
16        theta_est);
17      VaR_non_location = GAtquantile(gamma, d_est, v_est, theta_est);
18      L = v_est / (v_est + (-VaR_non_location * theta_est)^d_est);
19      a_num = v_est - (1/d_est);
20      b_num = 2/d_est;
21      a_den = v_est;
22      b_den = 1/d_est;
23      B_L_num = betainc(L, a_num, b_num) * beta(a_num, b_num);
24      B_L_den = betainc(L, a_den, b_den) * beta(a_den, b_den);
25      ES_values(b) = mu_est + c_est * (((-1)*v_est^(1/d_est))*(1+theta_est^2))
        / ((1 + theta_est^2) * theta_est)) * (B_L_num / B_L_den);
26   end
27 end

```

Listing 30: MATLAB Code for Bootstrap assuming Noncentral Student's t (NCT)

```

1 function [VaR_values, ES_values] = compute_ES_parametric_bootstrap_NCT(data, B, n,
2   gamma)
3   VaR_values = zeros(B, 1);
4   ES_values = zeros(B, 1);
5   for b = 1:B
6       fprintf('Starting NCT estimation for bootstrap sample %d out of %d...\n',
7         b, B);
8       ind = unidrnd(n, [n, 1]);
9       bootsamp = data(ind);
10      [param_est, ~, ~, ~, ~] = NCTestimation(bootsamp);
11      nu_hat = param_est(1);
12      delta_hat = param_est(2);
13      mu_hat = param_est(3);
14      sigma_hat = param_est(4);
15      VaR_standard = nctinv(gamma, nu_hat, delta_hat);
16      VaR_values(b) = mu_hat + sigma_hat * VaR_standard;
17      ES_values(b) = compute_ES_NCT(nu_hat, delta_hat, mu_hat, sigma_hat, gamma)
        ;
18   end
19 end

```

Listing 31: MATLAB Code for Non-parametric Bootstrap

```

1 function [VaR_values, ES_values] = compute_nonparametric_bootstrap_single(data, B,
2   n, gamma)
3   VaRlevel = gamma;

```

```

3     ESlevel = gamma;
4     VaR_values = zeros(B, 1);
5     ES_values = zeros(B, 1);
6     for b = 1:B
7         ind = unidrnd(n, [n, 1]);
8         bootsamp = data(ind);
9         VaR_bootstrap_np = quantile(bootsamp, VaRlevel);
10        VaR_values(b) = VaR_bootstrap_np;
11        ES_values(b) = mean(bootsamp(bootsamp <= VaR_bootstrap_np));
12    end
13 end

```

MATLAB Code for Part I Task 1: MLE for all used Distributions

Listing 32: MATLAB Code: MLE for Student's t

```

1 mle_params = mle(bootsamp, 'distribution', 'tLocationScale');
2 nu_mle = mle_params(3);
3 location_mle = mle_params(1);
4 scale_mle = mle_params(2);

```

Listing 33: MATLAB Code: MLE for Gaussian

```

1 [mu_hat, sigma_hat] = normfit(bootsamp);

```

Listing 34: MATLAB Code: MLE for Mixture of Normals

```

1 gm = fitgmdist(bootsamp, 2, 'Replicates', 5, 'Options', statset('MaxIter', 1000));

```

Listing 35: MATLAB Code: MLE for GAt

```

1 function [param, stderr, iters, loglik, Varcov] = GAtestimation(x, vlo, initvec,
    fixd)
2 if nargin < 2, vlo = []; end
3 if nargin < 3, initvec = []; end
4 if nargin < 4, fixd = []; end
5 if isempty(vlo), vlo = 0.01; end
6 if isempty(initvec)
7     versuch = 3; vhi = 4; loglik = -Inf;
8     vvec = linspace(vlo + 0.02, vhi, versuch);
9     for i = 1:versuch
10        vv = vvec(i);
11        if isempty(fixd)
12            initvec = [2 vv 0.98 0 3];
13        else
14            initvec = [vv 0.98 0 3];
15        end
16        [param0, stderr0, iters0, loglik0, Varcov0] = GAtestimation(x, vlo,
            initvec, fixd);
17        if loglik0 > loglik
18            loglik = loglik0;
19            param = param0; stderr = stderr0; iters = iters0; Varcov = Varcov0;
20        end
21    end
22    return
23 end

```

```

24 if isempty(fixd)
25     bound.lo = [0.1 vlo 0.2 -1 1e-4];
26     bound.hi = [30 100 3 2 1e+4];
27     bound.which = [1 1 1 0 1];
28 else
29     bound.lo = [vlo 0.2 -1 1e-4];
30     bound.hi = [100 3 2 1e+4];
31     bound.which = [1 1 0 1];
32 end
33 nob = length(x);
34 maxiter = length(initvec) * 100;
35 tol = 1e-8;
36 MaxFunEvals = length(initvec) * 400;
37 opts = optimset('Display', 'none', 'Maxiter', maxiter, 'TolFun', tol, 'TolX', tol,
    'MaxFunEvals', MaxFunEvals, 'LargeScale', 'Off');
38 if 1 == 1
39     [pout, fval, exitflag, theoutput, grad, hess] = ...
40         fminunc(@(param) GAtloglik(param, x, fixd, bound), einschrk(initvec, bound
            ), opts);
41 else
42     [pout, fval, exitflag, theoutput] = ...
43         fminsearch(@(param) GAtloglik(param, x, fixd, bound), einschrk(initvec,
            bound), opts);
44     hess = eye(length(pout));
45 end
46 V = inv(hess) / nob;
47 [param, V] = einschrk(pout, bound, V);
48 param = param'; Varcov = V;
49 stderr = sqrt(diag(V));
50 loglik = -fval * nob;
51 iters = theoutput.iterations;
52 function ll = GAtloglik(param, x, fixd, bound)
53 if nargin < 4, bound = 0; end
54 if isstruct(bound)
55     paramvec = einschrk(real(param), bound, 999);
56 else
57     paramvec = param;
58 end
59 if isempty(fixd)
60     d = paramvec(1); v = paramvec(2); theta = paramvec(3); mu = paramvec(4); c =
        paramvec(5);
61 else
62     d = fixd; v = paramvec(1); theta = paramvec(2); mu = paramvec(3); c = paramvec
        (4);
63 end
64 z = (x - mu) / c;
65 pdf = GAt(z, d, v, theta) / c;
66 llvec = log(pdf);
67 ll = -mean(llvec);
68 if isinf(ll), ll = 1e5; end
69
70 end
71 end
72 function [y, V_out] = einschrk(x, bound, V_in)
73 y = x;
74 for i = 1:length(x)
75     if y(i) < bound.lo(i)
76         y(i) = bound.lo(i);
77     elseif y(i) > bound.hi(i)

```

```

78         y(i) = bound.hi(i);
79     end
80 end
81 if nargin == 3
82     V_out = V_in;
83     for i = 1:length(y)
84         if bound.which(i) == 0
85             V_out(i, :) = 0;
86             V_out(:, i) = 0;
87         end
88     end
89 else
90     V_out = [];
91 end
92 end

```

Listing 36: MATLAB Code: MLE for Noncentral Student's t

```

1  function [param, stderr, iters, loglik, Varcov] = NCTestimation(x, initvec)
2  if nargin < 2 || isempty(initvec)
3      nu_init = 5;
4      delta_init = 0;
5      mu_init = mean(x);
6      sigma_init = std(x);
7      initvec = [nu_init, delta_init, mu_init, sigma_init];
8  end
9  bound.lo = [1e-3, -Inf, -Inf, 1e-3];
10 bound.hi = [100, Inf, Inf, Inf];
11 bound.which = [1, 0, 0, 1];
12 nob = length(x);
13 maxiter = length(initvec) * 100;
14 tol = 1e-8;
15 MaxFunEvals = length(initvec) * 400;
16 opts = optimset('Display', 'none', 'MaxIter', maxiter, 'TolFun', tol, 'TolX', tol,
17     'MaxFunEvals', MaxFunEvals, 'LargeScale', 'Off');
18 [pout, fval, exitflag, output, grad, hess] = fminunc(@(param) NCTnegloglik(param,
19     x, bound), einschrk(initvec, bound), opts);
20 if size(hess, 1) ~= size(hess, 2)
21     Varcov = NaN;
22     stderr = NaN(size(pout));
23 else
24     Varcov = inv(hess) / nob;
25     [param, Varcov] = einschrk(pout, bound, Varcov);
26     stderr = sqrt(diag(Varcov));
27 end
28 param = param';
29 loglik = -fval * nob;
30 iters = output.iterations;
31 end
32 function nll = NCTnegloglik(param, x, bound)
33 if isstruct(bound)
34     paramvec = einschrk(real(param), bound, 999);
35 else
36     paramvec = param;
37 end
38 nu = paramvec(1);
39 delta = paramvec(2);
40 mu = paramvec(3);
41 sigma = paramvec(4);

```

```
40
41 if nu <= 0 || sigma <= 0
42     nll = 1e5;
43     return;
44 end
45 z = (x - mu) / sigma;
46 ll = log(nctpdf(z, nu, delta)) - log(sigma);
47 if any(~isfinite(ll))
48     nll = 1e5;
49 else
50     nll = -mean(ll);
51 end
52 end
```


MATLAB Code for Task I Task 3: ES and VaR for Asymmetric Stable Paretian

Listing 37: MATLAB Code for VaR and ES visualization for Asymmetric Stable Paretian

```
1 alpha = 1.7; % Stability parameter (0 < alpha <= 2)
2 beta = -0.6; % Skewness parameter (-1 <= beta <= 1)
3 scale = 1; % Scale parameter (gamma > 0)
4 delta = 0; % Location parameter (real number)
5 gamma = 0.01;
6 [ES_true, VaR_true] = asymstableES(gamma, alpha, beta, delta, scale, 1);
7 fprintf('True Value at Risk (VaR): %.4f\n', VaR_true);
8 fprintf('True Expected Shortfall (ES): %.4f\n', ES_true);
9 x_min = -18; % Set x_min to -14
10 x_max = -5; % Set x_max to -4
11 x_values = linspace(x_min, x_max, 1000)'; % Ensure x_values is a column vector
12 [pdf_values, ~] = asymstabil((x_values - delta) / scale, alpha, beta);
13 pdf_values = pdf_values / scale; % Adjust for scale parameter
14 scale_factor = 1e5;
15 pdf_values_scaled = pdf_values * scale_factor;
16 figure;
17 plot(x_values, pdf_values_scaled, 'b-', 'LineWidth', 1.5, 'DisplayName', 'PDF');
18 hold on;
19 x_fill = [x_min; x_values(x_values <= VaR_true); VaR_true];
20 y_fill = [0; pdf_values_scaled(x_values <= VaR_true); 0];
21 fill(x_fill, y_fill, 'r', 'FaceAlpha', 0.3, 'EdgeColor', 'none', 'DisplayName', '
    VaR Region');
22 y_VaR = interp1(x_values, pdf_values_scaled, VaR_true);
23 y_ES = interp1(x_values, pdf_values_scaled, ES_true);
24 plot([VaR_true, VaR_true], [0, y_VaR], 'k--', 'LineWidth', 2, 'DisplayName', 'VaR'
    );
25 plot([ES_true, ES_true], [0, y_ES], 'g--', 'LineWidth', 2, 'DisplayName', 'ES');
26 text(VaR_true, y_VaR, sprintf('VaR = %.4f', VaR_true), 'HorizontalAlignment', '
    right', 'VerticalAlignment', 'bottom', 'FontSize', 10, 'Color', 'k');
27 text(ES_true, y_ES, sprintf('ES = %.4f', ES_true), 'HorizontalAlignment', 'left',
    'VerticalAlignment', 'top', 'FontSize', 10, 'Color', 'g');
28 xlabel('Value');
29 ylabel(['Density ', num2str(scale_factor)]);
30 title('PDF of Asymmetric Stable Paretian Distribution with VaR and ES');
31 legend('Location', 'NorthEast');
32 grid on;
33 hold off;
```

MATLAB Code for Part I Bonus: MLE for Asymmetric Stable Paretian

Listing 38: MATLAB Code for MLE of Asymmetric Stable Paretian Distributions

```
1 function [param, stderr, iters, loglik, Varcov] = ASPEstimation(data, vlo, initvec
   , fixd)
2 % data: Observed data to fit.
3 % vlo: Minimum value for the stability parameter alpha.
4 % initvec: Initial guess for parameters [alpha, beta, gamma, delta].
5 % fixd: Fixed delta parameter (optional).
6     if nargin < 2, vlo = 0.1; end
7     if nargin < 3, initvec = []; end
8     if nargin < 4, fixd = []; end
9     if isempty(initvec)
10         initvec = [1.5, 0, 1, 0];
11     end
12     if isempty(fixd)
13         bound.lo = [vlo, -2, 1e-4, -1];
14         bound.hi = [30, 0.1, 1e4, 2];
15         bound.which = [1, 1, 1, 0];
16     else
17         bound.lo = [vlo, -1, 1e-4];
18         bound.hi = [2, 1, 1e4];
19         bound.which = [1, 1, 1];
20     end
21     opts = optimset('Display', 'off', 'MaxIter', 500, 'TolFun', 1e-8, ...
22         'TolX', 1e-8, 'MaxFunEvals', 2000);
23     [pout, fval, exitflag, output, grad, hess] = fminunc(...
24         @(param) ASPLogLik(param, data, fixd, bound), ...
25         einschraenk(initvec, bound), opts);
26     nobs = length(data);
27     V = inv(hess) / nobs;
28     [param, V] = einschraenk(pout, bound, V);
29     param = param';
30     Varcov = V;
31     stderr = sqrt(diag(V));
32     loglik = -fval * nobs;
33     iters = output.iterations;
34 end
35 function ll = ASPLogLik(param, data, fixd, bound)n
36     if nargin < 4, bound = 0; end
37     if isstruct(bound)
38         paramvec = einschraenk(real(param), bound, 999);
39     else
40         paramvec = param;
41     end
42     if isempty(fixd)
43         alpha = paramvec(1);
44         beta = paramvec(2);
45         gamma = paramvec(3);
46         delta = paramvec(4);
47     else
48         alpha = paramvec(1);
49         beta = paramvec(2);
50         gamma = paramvec(3);
51         delta = fixd;
52     end
```

```

53     z = (data - delta) / gamma;
54     pdf = ASPPDF(z, alpha, beta) / gamma;
55     llvec = log(pdf);
56     ll = -mean(llvec);
57     if isinf(ll), ll = 1e5; end
58 end
59 function pdf = ASPPDF(x, alpha, beta)
60     tol = 1e-8;
61     lo = tol;
62     hi = 1 - tol;
63     pdf = zeros(size(x));
64     for i = 1:length(x)
65         xi = x(i);
66         pdf(i) = integral(@(u) ASPIntHelper(u, xi, alpha, beta), lo, hi, ...
67             'RelTol', 1e-6, 'AbsTol', 1e-8) / pi;
68     end
69 end
70 function I = ASPIntHelper(u, x, alpha, beta)
71     t = (1 - u) ./ u;
72     if alpha == 1
73         cf = exp(-abs(t) .* (1 + 1i * beta * (2 / pi) .* sign(t) .* log(t)));
74     else
75         cf = exp(-(abs(t).^alpha) .* (1 - 1i * beta * sign(t) * tan(pi * alpha /
76             2)));
77     end
78     z = exp(-1i * t * x) .* cf;
79     I = real(z) .* u.^(-2);
80 end
81 function [y, V_out] = einschraenk(x, bound, V_in)
82     y = x;
83     for i = 1:length(x)
84         if y(i) < bound.lo(i)
85             y(i) = bound.lo(i);
86         elseif y(i) > bound.hi(i)
87             y(i) = bound.hi(i);
88         end
89     end
90     if nargin == 3 && ~isempty(V_in)
91         V_out = V_in;
92         for i = 1:length(y)
93             if bound.which(i) == 0
94                 V_out(i, :) = 0;
95                 V_out(:, i) = 0;
96             end
97         end
98     else
99         V_out = [];
100     end
end

```

Listing 39: MATLAB Code for Asymmetric Stable Paretian Bootstrap

```

1 function [VaR_values, ES_values] = compute_ES_Pareto_Bootstrap(data, B, n, gamma)
2     VaR_values = zeros(B, 1);
3     ES_values = zeros(B, 1);
4     for b = 1:B
5         ind = unidrnd(n, [n, 1]);
6         bootsamp = data(ind);
7         initvec = [1.5, 0, 1, 0];
8         vlo = 0.1;
9         fixd = [];
10        [param, ~, ~, ~, ~] = ASPEstimation(bootsamp, vlo, initvec, fixd);
11        estimates = param;
12        alpha = estimates(1);
13        beta = estimates(2);
14        scale = estimates(3);
15        location = estimates(4);
16        fprintf('Parameters %.4f\n', param);
17        [actual1ES, actual2VaR] = asymstableES(gamma, alpha, beta, location, scale
18            , 1) %asymstablesES function given in report;
19        VaR_values(b) = actual2VaR;
20        ES_values(b) = actual1ES;
21        if isnan(ES_values(b)) || isnan(VaR_values(b))
22            ES_values(b) = mean(bootsamp(bootsamp <= VaR_values(b)));
23            ES_values(b) = actual1;
24        end
25    end
end

```

MATLAB Code for Task II Task 1: Mixture-density and Sample Data for Bivariate Laplace

Listing 40: MATLAB Code for Simulating and Plotting Sample Data for Mixture of two Bivariate Laplace Distributions

```
1 function simulate_bivariate_mixture_laplace()
2 n = 10000;
3 d = 2;
4 lambda = [0.7, 0.3];
5 mu1 = [0; 0];
6 Sigma1 = [1, 0.5; 0.5, 1];
7 b1 = 10;
8 mu2 = [0; 0];
9 Sigma2 = [9, 2; 2, 9];
10 b2 = 5;
11 if min(eig(Sigma1)) <= 0 || min(eig(Sigma2)) <= 0
12     error('Covariance matrices must be positive definite.');
```

```
13 end
14 u = rand(n, 1);
15 C = ones(n, 1);
16 C(u > lambda(1)) = 2;
17 Y = zeros(n, d);
18 idx1 = (C == 1);
19 n1 = sum(idx1);
20 G1 = gamrnd(b1, 1, n1, 1);
21 Z1 = randn(n1, d);
22 L1 = chol(Sigma1, 'lower');
23 Y1 = repmat(mu1', n1, 1) + (sqrt(G1) * ones(1, d)) .* (Z1 * L1');
24 idx2 = (C == 2);
25 n2 = sum(idx2);
26 G2 = gamrnd(b2, 1, n2, 1);
27 Z2 = randn(n2, d);
28 L2 = chol(Sigma2, 'lower');
29 Y2 = repmat(mu2', n2, 1) + (sqrt(G2) * ones(1, d)) .* (Z2 * L2');
30 Y(idx1, :) = Y1;
31 Y(idx2, :) = Y2;
32 num_bins = [50 50];
33 [counts, centers] = hist3(Y, 'Nbins', num_bins);
34 dx = centers{1}(2) - centers{1}(1);
35 dy = centers{2}(2) - centers{2}(1);
36 joint_density = counts / (sum(counts(:)) * dx * dy);
37 [X_grid, Y_grid] = meshgrid(centers{1}, centers{2});
38 figure;
39 surf(X_grid, Y_grid, joint_density, 'EdgeColor', 'none');
40 xlabel('Y_1', 'FontSize', 12);
41 ylabel('Y_2', 'FontSize', 12);
42 zlabel('Estimated Density', 'FontSize', 12);
43 title('Estimated Joint Density of the Bivariate Mixture Laplace Distribution', '
    FontSize', 14);
44 shading interp;
45 colormap parula;
46 colorbar;
47 view(-30, 45);
48 end
```

MATLAB Code for Task II Task 2: Bessel Function and MLE for discrete mixture of bivariate Laplace

See Listing 12 and Listing 13.

MATLAB Code for Task II Task 2b

Listing 41: MATLAB Code for finding Optimal Guesses in MLE for Discrete Mixture of two Bivariate Laplace Distributions

```
1 function initial_params = optimize_initial_values(data, k)
2     [d, n] = size(data);
3     assert(d == 2, 'This function supports only bivariate data. ');
4     lambda_init = ones(1, k) / k;
5     [cluster_idx, centroids] = kmeans(data, k, 'Replicates', 10, 'Distance', '
        sqeuclidean');
6     mu_init = centroids';
7     Sigma_list = cell(k, 1);
8     for i = 1:k
9         cluster_data = data(:, cluster_idx == i);
10        if size(cluster_data, 2) > 1
11            Sigma_list{i} = cov(cluster_data');
12        else
13            Sigma_list{i} = eye(d) * 0.1;
14        end
15    end
16    b_init = zeros(1, k);
17    for i = 1:k
18        cluster_data = data(:, cluster_idx == i);
19        if ~isempty(cluster_data)
20            b_init(i) = var(cluster_data(:)) / mean(abs(cluster_data(:)));
21        else
22            b_init(i) = 1;
23        end
24    end
25    log_b_init = log(b_init);
26    cov_params_init = [];
27    for i = 1:k
28        [L, ~] = chol(Sigma_list{i}, 'lower');
29        cov_params_init = [cov_params_init, log(diag(L)')];
30        if d == 2
31            cov_params_init = [cov_params_init, L(2, 1)];
32        end
33    end
34    gamma_init = log(lambda_init(1:(k-1)) ./ lambda_init(k));
35    param_init = [gamma_init, mu_init(:)', cov_params_init, log_b_init];
36    initial_params = struct();
37    initial_params.lambda = lambda_init;
38    initial_params.mu = mu_init;
39    initial_params.Sigma = Sigma_list;
40    initial_params.b = b_init;
41    initial_params.param_vector = param_init;
```

MATLAB Code for Task II Task 3: MLE for Bivariate NCT

Listing 42: MATLAB Code ES and VaR Visualization (Paretian)

```

1 function pdfln = mvnctpdfln(x, gam, R, v)
2     [d, T] = size(x);
3     [R_chol, err] = cholcov(R, 0);
4     assert(err == 0, 'R is not positive definite');
5     vn2 = (v + d) / 2;
6     xm = x;
7     rho = sum((R_chol \ xm).^2, 1);
8     pdfln = gammaln(vn2) - d/2 * log(pi * v) - gammaln(v / 2) - ...
9         sum(log(diag(R_chol))) - vn2 * log1p(rho / v);
10    if all(gam == 0)
11        return;
12    end
13    gam = reshape(gam, [], 1);
14    gam_proj = R_chol \ gam;
15    gmg = sum(gam_proj.^2);
16    pdfln = pdfln - 0.5 * gmg + log(sum(gam_proj));
17 end
18 function [param, stderr, iters, loglik, Varcov] = MVNCT2estimation(data)
19     [d, T] = size(data);
20     assert(d == 2, 'Only supports 2-dimensional data');
21     bound.lo = [1.1, -1, -1, 0.01, 0.01, -1, -4, -4];
22     bound.hi = [20, 1, 1, 100, 100, 1, 4, 4];
23     bound.which = [1, 0, 0, 1, 1, 1, 1, 1];
24     mu_init = mean(data, 2);
25     scale_init = std(data, 0, 2);
26     correlation_init = corr(data(1, :)', data(2, :));
27     noncentrality_init = [0.1, 0.1];
28     initvec = [3, mu_init', scale_init', correlation_init, noncentrality_init];
29     initvec = einschraenk(initvec, bound);
30     maxiter = 300;
31     tol = 1e-6;
32     opts = optimset('Display', 'iter', 'MaxIter', maxiter, 'TolFun', tol, 'TolX',
33         tol, 'LargeScale', 'off');
34     obj_fun = @(param) MVNCTloglik(param, data, bound);
35     [pout, fval, ~, output, ~, hess] = fminunc(obj_fun, initvec, opts);
36     [param, Varcov] = einschraenk(pout, bound, inv(hess) / T);
37     stderr = sqrt(diag(Varcov));
38     loglik = -fval * T;
39     iters = output.iterations;
40 end
41 function ll = MVNCTloglik(param, data, bound)
42     k = param(1); mu = param(2:3); scale = param(4:5); gam = param(7:8);
43     R12 = param(6); R = [1, R12; R12, 1];
44     if min(eig(R)) < 1e-4
45         ll = 1e5;
46         return;
47     end
48     data_scaled = data;
49     for i = 1:2
50         data_scaled(i, :) = (data(i, :) - mu(i)) / scale(i);
51     end
52     llvec = mvnctpdfln(data_scaled, gam, R, k) - log(prod(scale));
53     ll = -mean(llvec);
54     if isinf(ll)
55         ll = 1e5;

```



```

55     end
56 end
57 function [y, V_out] = einschraenk(x, bound, V_in)
58     y = x;
59     for i = 1:length(x)
60         if y(i) < bound.lo(i)
61             y(i) = bound.lo(i);
62         elseif y(i) > bound.hi(i)
63             y(i) = bound.hi(i);
64         end
65     end
66     if nargin == 3 && ~isempty(V_in)
67         V_out = V_in;
68         for i = 1:length(y)
69             if bound.which(i) == 0
70                 V_out(i, :) = 0;
71                 V_out(:, i) = 0;
72             end
73         end
74     else
75         V_out = [];
76     end
77 end
78 function simulate_and_estimate()
79     d = 2;
80     T = 10000;
81     v = 13.38;
82     mu = [-0.07; -0.07];
83     scale = [2.59, 5.22];
84     R = [1, 0.77; 0.77, 1];
85     [R_chol, err] = chol(R, 'lower');
86     assert(err == 0, 'R is not positive definite');
87     z = randn(d, T);
88     chi2_samples = chi2rnd(v, 1, T);
89     x = R_chol * z;
90     x = bsxfun(@times, x, sqrt(v ./ chi2_samples));
91     x = bsxfun(@plus, x, mu);
92     for i = 1:d
93         x(i, :) = x(i, :) * scale(i);
94     end
95     [param, stderr, iters, loglik, Varcov] = MVNCT2estimation(x);
96 end
97 function plot_sample_data_and_contours(x, param)
98     v = param(1);
99     mu = param(2:3)';
100    scale = param(4:5)';
101    R12 = param(6);
102    R = [1, R12; R12, 1];
103    [X, Y] = meshgrid(linspace(min(x(1, :)), max(x(1, :)), 100), ...
104                      linspace(min(x(2, :)), max(x(2, :)), 100));
105    grid_points = [X(:), Y(:)]';
106    for i = 1:2
107        grid_points(i, :) = (grid_points(i, :) - mu(i)) / scale(i);
108    end
109    pdf_values = mvnctpdfln(grid_points, param(7:8), R, v);
110    pdf_values = reshape(exp(pdf_values), size(X));
111    figure;
112    scatter(x(1, :), x(2, :), 1, [0.6, 0.6, 0.6], 'filled');
113    hold on;

```

```
114     contour(X, Y, pdf_values, 10, 'LineWidth', 1.5);
115     title('Sample Data with Estimated Contours');
116     xlabel('X1');
117     ylabel('X2');
118     legend({'Sample Data', 'Estimated Contours'});
119     grid on;
120     hold off;
121 end
122 simulate_and_estimate();
```

MATLAB Code for Task II Task 4: Pair Selection and MLE in DJIA Data

Listing 43: MATLAB Code for AIC and BIC for MVNCT fit on DJIA Data

```

1 AIC_matrix = zeros(50,2);
2 BIC_matrix = zeros(50,1);
3
4 for sim = 1:50
5     simulate_and_estimate();
6     AIC_matrix(sim,:) = simulate_and_estimate();
7 end
8
9 function pdfln = mvnctpdfln(x, mu, gam, v, Sigma)
10     [d, t] = size(x);
11     C = Sigma;
12     [R, err] = cholcov(C, 0);
13     assert(err == 0, 'C is not (semi) positive definite');
14     mu = reshape(mu, length(mu), 1);
15     gam = reshape(gam, length(gam), 1);
16     vn2 = (v + d) / 2;
17     xm = x - repmat(mu, 1, t);
18     rho = sum((R' \ xm).^2, 1);
19     pdfln = gammaln(vn2) - d / 2 * log(pi * v) - gammaln(v / 2) - ...
20         sum(slog(diag(R))) - vn2 * log1p(rho / v);
21     if all(gam == 0), return; end
22     idx = (pdfln >= -37);
23     maxiter = 1e4;
24     k = 0;
25     if any(idx)
26         gcg = sum((R' \ gam).^2);
27         pdfln = pdfln - 0.5 * gcg;
28         xcg = xm' * (C \ gam);
29         term = 0.5 * log(2) + log(max(xcg, realmin)) - 0.5 * slog(v + rho');
30         term(term == -inf) = log(realmin);
31         term(term == +inf) = log(realmax);
32         logterms = gammaln((v + d + k) / 2) - gammaln(k + 1) - gammaln(vn2) + k *
            term;
33         ff = exp(logterms - max(logterms));
34         logsumk = log(ff);
35         while k < maxiter
36             k = k + 1;
37             logterms = gammaln((v + d + k) / 2) - gammaln(k + 1) - gammaln(vn2) +
                k * term(idx);
38             ff = real(exp(logterms - logsumk(idx)));
39             logsumk(idx) = logsumk(idx) + log1p(ff);
40             idx(idx) = (abs(ff) > 1e-4);
41             if all(idx == false), break, end
42         end
43         pdfln = real(pdfln + logsumk');
44     end
45 end
46
47 function y = slog(x)
48     y = log(max(realmin, min(realmax, x)));
49 end
50
51 function [param, stderr, iters, loglik, Varcov] = MVNCT2estimation(data)

```

```

52     [d, T] = size(data);
53     assert(d == 2, 'Only supports 2-dimensional data');
54     bound.lo = [1.1, -30, -30, 0.01, 0.01, -1, -4, -4];
55     bound.hi = [100, 30, 30, 100, 100, 1, 4, 4];
56     bound.which = [1, 0, 0, 1, 1, 0, 1, 1];
57     mu_init = mean(data, 2);
58     scale_init = std(data, 0, 2);
59     correlation_init = corr(data(1, :)', data(2, :))';
60     noncentrality_init = [0.1, 0.1];
61     initvec = [3, mu_init', scale_init', correlation_init, noncentrality_init];
62     initvec = einschraenk(initvec, bound);
63     opts = optimset('Display', 'off', 'MaxIter', 300, 'TolFun', 1e-8, ...
64                   'TolX', 1e-8, 'MaxFunEvals', 2000, 'Display', 'iter');
65     obj_fun = @(param) MVNCTloglik(param, data, bound);
66     [pout, fval, ~, output, ~, hess] = fminunc(obj_fun, initvec, opts);
67     [param, Varcov] = einschraenk(pout, bound, inv(hess) / T);
68     stderr = sqrt(diag(Varcov));
69     loglik = -fval * T;
70     iters = output.iterations;
71 end
72
73 function ll = MVNCTloglik(param, x, bound)
74     if nargin < 3, bound = 0; end
75     if isstruct(bound), param = einschraenk(real(param), bound, 999); end
76     k = param(1);
77     mu = param(2:3);
78     scale = param(4:5);
79     gam = param(7:8);
80     R12 = param(6);
81     R = [1 R12; R12 1];
82     if min(eig(R)) < 1e-4
83         ll = 1e5;
84     else
85         xx = x;
86         for i = 1:2, xx(i,:) = (x(i,:) - mu(i)) / scale(i); end
87         llvec = mvnctpdfln(xx, mu, gam, k, R) - log(prod(scale));
88         ll = -mean(llvec);
89         if isinf(ll), ll = 1e5; end
90     end
91 end
92
93 function [y, V_out] = einschraenk(x, bound, V_in)
94     y = x;
95     for i = 1:length(x)
96         if y(i) < bound.lo(i)
97             y(i) = bound.lo(i);
98         elseif y(i) > bound.hi(i)
99             y(i) = bound.hi(i);
100         end
101     end
102     if nargin == 3 && ~isempty(V_in)
103         V_out = V_in;
104         for i = 1:length(y)
105             if bound.which(i) == 0
106                 V_out(i, :) = 0;
107                 V_out(:, i) = 0;
108             end
109         end
110     else

```

```

111         V_out = [];
112     end
113 end
114
115 function [AIC, BIC] = simulate_and_estimate()
116     x = DJIA_stock_selection();
117     n = size(x, 1);
118     [param, stderr, iters, loglik, Varcov] = MVNCT2estimation(x);
119     num_params_nct = length(param);
120     loglik_nct = loglik;
121     AIC = 2 * num_params_nct - 2 * loglik_nct;
122     BIC = num_params_nct * log(n) - 2 * loglik_nct;
123 end
124
125 function dataXX = DJIA_stock_selection()
126     fileName = 'DJIA30stockreturns.mat';
127     data = load(fileName);
128     data = data.DJIARet;
129     returns = data;
130     corrMatrix = corr(returns);
131     corrMatrix(logical(eye(size(corrMatrix)))) = -Inf;
132     [row, col] = find(triu(corrMatrix, 1));
133     probs = abs(corrMatrix(sub2ind(size(corrMatrix), row, col)));
134     probs = probs / sum(probs);
135     selectedIdx = randsample(length(probs), 1, true, probs);
136     pair = [row(selectedIdx), col(selectedIdx)];
137     pairData = data(:, pair);
138     dataX = pairData;
139     dataXX = dataX.';
140 end

```

Listing 44: MATLAB Code for AIC and BIC for DJIA Data, MVNCT and Mixture of Laplace fit on DJIA Data

```

1 AIC_matrix = zeros(50,1);
2 BIC_matrix = zeros(50,1);
3 AIC_matrixNCT = zeros(50,1);
4 BIC_matrixNCT = zeros(50,1);
5
6 for sim = 1:1
7     [AIC_matrix(sim,:), BIC_matrix(sim,:), AIC_matrixNCT(sim,:), BIC_matrixNCT(sim,
8         ,:)] = run_it();
9 end
10
11 function [samples, C] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda, n)
12     k = length(lambda);
13     d = length(mu_list{1});
14     if length(mu_list) ~= k || length(Sigma_list) ~= k || length(b_list) ~= k
15         error('mu_list, Sigma_list, and b_list must all have length equal to the
16             number of components k. ');
17     end
18     if abs(sum(lambda) - 1) > 1e-6
19         error('Mixture weights lambda must sum to 1. ');
20     end
21     samples = zeros(d, n);
22     C = zeros(1, n);
23     lambda_cum = cumsum(lambda);
24     U = rand(1, n);
25     C = discretize(U, [0; lambda_cum(:)]);

```

```

24     for c = 1:k
25         idx_c = find(C == c);
26         n_c = length(idx_c);
27         if n_c > 0
28             mu_c = mu_list{c};
29             Sigma_c = Sigma_list{c};
30             b_c = b_list(c);
31             G_c = gamrnd(b_c, 1, [1, n_c]);
32             Z_c = mvnrnd(zeros(1, d), Sigma_c, n_c)';
33             Y_c = mu_c + Z_c .* sqrt(G_c);
34             samples(:, idx_c) = Y_c;
35         end
36     end
37 end
38
39 function f = laplacepdf(x, mu, Sigma, b)
40     d = length(mu);
41     diff = x - mu;
42     [~, p] = chol(Sigma, 'lower');
43     if p > 0
44         Sigma = Sigma + 1e-6 * eye(size(Sigma));
45     end
46     try
47         [L, p] = chol(Sigma, 'lower');
48     catch
49         [L, p] = chol(pinv(Sigma), 'lower');
50     end
51     if p > 0
52         f = realmin;
53         return;
54     end
55     z = L \ diff;
56     m = sum(z.^2);
57     log_det_Sigma = 2 * sum(log(diag(L)));
58     log_const = log(2) - gammaln(max(b, realmin)) - 0.5 * (log_det_Sigma + d * log
        (2 * pi)) + (b / 2 - d / 4) * log(max(m / 2, realmin));
59     nu = b - d / 2;
60     sqrt2m = sqrt(2 * m);
61     log_K = log(besselk(nu, sqrt2m) + realmin);
62     log_f = log_const + log_K;
63     f = exp(log_f);
64     if isnan(f) || isinf(f)
65         f = realmin;
66     end
67 end
68
69 function negloglik = negloglik_laplace(param, data, k)
70     [d, n] = size(data);
71     gamma = param(1:(k-1));
72     lambda = [exp(gamma) ./ (1 + sum(exp(gamma))), 1 / (1 + sum(exp(gamma)))];
73     mu = reshape(param(k:(k + k * d - 1)), [d, k]);
74     cov_params = param((k + k * d):(k + k * d + k * 3 - 1));
75     b = exp(param((k + k * d + k * 3):end));
76     Sigma_list = cell(k, 1);
77     idx = 1;
78     for i = 1:k
79         L_vec = cov_params(idx:(idx + 2));
80         idx = idx + 3;
81         L = [exp(L_vec(1)), 0; L_vec(2), exp(L_vec(3))];

```

```

82     Sigma_list{i} = L * L';
83 end
84 ll = 0;
85 for i = 1:n
86     xi = data(:, i);
87     density_i = 0;
88     for j = 1:k
89         density_ij = lambda(j) * laplacepdf(xi, mu(:, j), Sigma_list{j}, b(j))
90         ;
91         density_i = density_i + density_ij;
92     end
93     ll = ll + log(density_i);
94 end
95 scale_factor = prod(std(data, 0, 2));
96 log_likelihood_adjustment = -n * log(scale_factor);
97 negloglik = -ll + log_likelihood_adjustment;
98 end
99 function [AIC_matrix, BIC_matrix, AIC_matrixNCT, BIC_matrixNCT] = run_it()
100 mu_list = {[2; 2], [-2; -2]};
101 Sigma_list = {[1, 0.5; 0.5, 1], [10, -0.3; -0.3, 10]};
102 b_list = [3, 2];
103 lambda_true = [0.4, 0.6];
104 n = 8000;
105 [data, ~] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda_true, n);
106 k = 2;
107 %data = DJIA_stock_selection_highest_cov();
108 %data = simulate_and_estimate(); Generates MVNCT sample data
109 initial_params = optimize_initial_values(data, k);
110 lambda_init = initial_params.lambda;
111 gamma_init = log(lambda_init(1:(k-1)) ./ lambda_init(k));
112 mu_init = initial_params.mu;
113 Sigma_list = initial_params.Sigma;
114 cov_params_init = [];
115 for i = 1:k
116     [L, ~] = chol(Sigma_list{i}, 'lower');
117     cov_params_init = [cov_params_init, log(diag(L)')];
118     if size(Sigma_list{i}, 1) == 2
119         cov_params_init = [cov_params_init, L(2, 1)];
120     end
121 end
122 b_init = initial_params.b;
123 param_init = [gamma_init, mu_init(:)', cov_params_init, b_init];
124 options = optimset('Display', 'off', 'MaxIter', 100, 'TolFun', 1e-8, 'TolX', 1
125     e-8, 'MaxFunEvals', 2000);
126 negloglik_fun = @(param) negloglik_laplace(param, data, k);
127 [param_est, fval] = fminunc(negloglik_fun, param_init, options);
128 loglik_laplace = -fval;
129 num_params_laplace = length(param_est);
130 n = size(data, 2);
131 aic_laplace = 2 * num_params_laplace - 2 * loglik_laplace;
132 bic_laplace = num_params_laplace * log(n) - 2 * loglik_laplace;
133 AIC_matrix = aic_laplace;
134 BIC_matrix = bic_laplace;
135 [param, ~, ~, loglik2, ~] = MVNCT2estimation(data);
136 num_params_NCT = length(param);
137 aic_NCT = 2 * num_params_NCT - 2 * loglik2;
138 bic_NCT = num_params_NCT * log(n) - 2 * loglik2;
139 AIC_matrixNCT = aic_NCT;

```

```

139     BIC_matrixNCT = bic_NCT;
140 end
141 function pdfln = mvnctpdfln(x, mu, gam, v, Sigma)
142 [d, t] = size(x);
143 C = Sigma;
144 [R, err] = cholcov(C, 0);
145 assert(err == 0, 'C is not (semi) positive definite');
146 mu = reshape(mu, length(mu), 1);
147 gam = reshape(gam, length(gam), 1);
148 vn2 = (v + d) / 2;
149 xm = x - repmat(mu, 1, t);
150 rho = sum((R' \ xm).^2, 1);
151 pdfln = gammaln(vn2) - d / 2 * log(pi * v) - gammaln(v / 2) - sum(slog(diag(R))) -
    vn2 * log1p(rho / v);
152 if all(gam == 0), return; end
153 idx = pdfln >= -37;
154 maxiter = 1e4;
155 k = 0;
156 if any(idx)
157     gcg = sum((R' \ gam).^2);
158     pdfln = pdfln - 0.5 * gcg;
159     xcg = xm' * (C \ gam);
160     term = 0.5 * log(2) + log(xcg) - 0.5 * slog(v + rho');
161     term(term == -inf) = log(realmin);
162     term(term == +inf) = log(realmax);
163     logterms = gammaln((v + d + k) / 2) - gammaln(k + 1) - gammaln(vn2) + k * term
        ;
164     ff = real(exp(logterms));
165     logsumk = log(ff);
166     while k < maxiter
167         k = k + 1;
168         logterms = gammaln((v + d + k) / 2) - gammaln(k + 1) - gammaln(vn2) + k *
            term(idx);
169         ff = real(exp(logterms - logsumk(idx)));
170         logsumk(idx) = logsumk(idx) + log1p(ff);
171         idx(idx) = abs(ff) > 1e-4;
172         if all(idx == false), break; end
173     end
174     pdfln = real(pdfln + logsumk');
175 end
176 end
177
178 function y = slog(x)
179 y = log(max(realmin, min(realmax, x)));
180 end
181
182 function [param, stderr, iters, loglik, Varcov] = MVNCT2estimation(data)
183 [d, T] = size(data);
184 assert(d == 2, 'Only supports 2-dimensional data');
185 bound.lo = [1.1, -30, -30, 0.01, 0.01, -1, -4, -4];
186 bound.hi = [100, 30, 30, 100, 100, 1, 4, 4];
187 bound.which = [1, 0, 0, 1, 1, 0, 1, 1];
188 mu_init = mean(data, 2);
189 scale_init = std(data, 0, 2);
190 correlation_init = corr(data(1, :)', data(2, :));
191 noncentrality_init = [0.1, 0.1];
192 initvec = [3, mu_init', scale_init', correlation_init, noncentrality_init];
193 initvec = einschraenk(initvec, bound);
194 opts = optimset('Display', 'off', 'MaxIter', 300, 'TolFun', 1e-8, 'TolX', 1e-8, '

```



```

195     'MaxFunEvals', 2000, 'Display', 'iter');
196 obj_fun = @(param) MVNCTloglik(param, data, bound);
197 [pout, fval, ~, output, ~, hess] = fminunc(obj_fun, initvec, opts);
198 [param, Varcov] = einschraenk(pout, bound, inv(hess) / T);
199 stderr = sqrt(diag(Varcov));
200 loglik = -fval * T;
201 iters = output.iterations;
202 end
203
204 function ll = MVNCTloglik(param, x, bound)
205 if nargin < 3, bound = 0; end
206 if isstruct(bound), param = einschraenk(real(param), bound, 999); end
207 k = param(1);
208 mu = param(2:3);
209 scale = param(4:5);
210 gam = param(7:8);
211 R12 = param(6);
212 R = [1 R12; R12 1];
213 if min(eig(R)) < 1e-4, ll = 1e5;
214 else
215     xx = x;
216     for i = 1:2, xx(i, :) = (x(i, :) - mu(i)) / scale(i); end
217     llvec = mvnctpdfn(xx, mu, gam, k, R) - log(prod(scale));
218     ll = -mean(llvec);
219     if isinf(ll), ll = 1e5; end
220 end
221 end
222
223 function [y, V_out] = einschraenk(x, bound, V_in)
224 y = x;
225 for i = 1:length(x)
226     if y(i) < bound.lo(i), y(i) = bound.lo(i);
227     elseif y(i) > bound.hi(i), y(i) = bound.hi(i);
228     end
229 end
230 if nargin == 3 && ~isempty(V_in)
231     V_out = V_in;
232     for i = 1:length(y)
233         if bound.which(i) == 0
234             V_out(i, :) = 0;
235             V_out(:, i) = 0;
236         end
237     end
238 else
239     V_out = [];
240 end
end

```

Listing 45: MATLAB Code for AIC and BIC for DJIA Data, MVNCT and Mixture of Laplace fit on DJIA Data

```

1 AIC_matrix = zeros(50,1);
2 BIC_matrix = zeros(50,1);
3 AIC_matrixNCT = zeros(50,1);
4 BIC_matrixNCT = zeros(50,1);
5 use_simulated_data = false;
6 for sim = 1:100
7     [AIC_matrix(sim,:), BIC_matrix(sim,:), AIC_matrixNCT(sim,:), BIC_matrixNCT(sim,
        ,:)] = run_it(use_simulated_data);

```

```

8 end
9 mean(abs(AIC_matrix)<abs(AIC_matrixNCT))
10 mean(abs(BIC_matrix)<abs(BIC_matrixNCT))
11 function [samples, C] = mixlaplacesim(mu_list, Sigma_list, b_list, lambda, n)
12     k = length(lambda);
13     d = length(mu_list{1});
14     samples = zeros(d, n);
15     C = zeros(1, n);
16     lambda_cum = cumsum(lambda);
17     U = rand(1, n);
18     C = discretize(U, [0; lambda_cum(:)]);
19     for c = 1:k
20         idx_c = find(C == c);
21         n_c = length(idx_c);
22         if n_c > 0
23             mu_c = mu_list{c};
24             Sigma_c = Sigma_list{c};
25             b_c = b_list(c);
26             G_c = gamrnd(b_c, 1, [1, n_c]);
27             Z_c = mvnrnd(zeros(1, d), Sigma_c, n_c)';
28             Y_c = mu_c + Z_c .* sqrt(G_c);
29             samples(:, idx_c) = Y_c;
30         end
31     end
32 end
33 function f = laplacepdf(x, mu, Sigma, b)
34     d = length(mu);
35     diff = bsxfun(@minus, x, mu);
36     [L, p] = chol(Sigma, 'lower');
37     if p > 0
38         f = realmin;
39         return;
40     end
41     z = L \ diff;
42     m = sum(z.^2, 1);
43     if b == (d + 1)/2
44         sqrt2m = sqrt(2 * m);
45         K_v = sqrt(pi ./ (2 * sqrt2m)) .* exp(-sqrt2m);
46         log_det_Sigma = 2 * sum(log(diag(L)));
47         log_const = log(2) - (d / 2) * log(2 * pi) - 0.5 * log_det_Sigma;
48         log_f = log_const - sqrt2m + 0.5 * log(pi ./ (2 * sqrt2m));
49         f = exp(log_f);
50     else
51         v = b - d / 2;
52         sqrt2m = sqrt(2 * m);
53         K_v = bessellk(v, sqrt2m);
54         K_v(K_v == Inf) = realmax;
55         K_v(isnan(K_v)) = realmin;
56         K_v(K_v == 0) = realmin;
57         log_det_Sigma = 2 * sum(log(diag(L)));
58         log_const = log(2) - gammaln(b) - (d / 2) * log(2 * pi) - 0.5 *
            log_det_Sigma + (v) * log(sqrt2m);
59         log_f = log_const + log(K_v);
60     end
61     f = exp(log_f);
62     f(isnan(f) | isinf(f) | f == 0) = realmin;
63 end
64 function negloglik = negloglik_laplace(param, data, k)
65     [d, n] = size(data);

```

```

66 gamma = param(1:(k-1));
67 lambda = [exp(gamma) ./ (1 + sum(exp(gamma))), 1 / (1 + sum(exp(gamma)))];
68 mu = reshape(param(k:(k + k * d - 1)), [d, k]);
69 cov_params = param((k + k * d):(k + k * d + k * 3 - 1));
70 b = exp(param((k + k * d + k * 3):end));
71 Sigma_list = cell(k, 1);
72 idx = 1;
73 for i = 1:k
74     L_vec = cov_params(idx:(idx + 2));
75     idx = idx + 3;
76     L = [exp(L_vec(1)), 0; L_vec(2), exp(L_vec(3))];
77     Sigma_list{i} = L * L';
78 end
79 log_lambda = log(lambda);
80 log_densities = zeros(k, n);
81 for j = 1:k
82     diff = bsxfun(@minus, data, mu(:, j));
83     [L, p] = chol(Sigma_list{j}, 'lower');
84     if p > 0
85         negloglik = Inf;
86         return;
87     end
88     z = L \ diff;
89     m = sum(z.^2, 1);
90     if b(j) == (d + 1)/2
91         sqrt2m = sqrt(2 * m);
92         log_K = 0.5 * (log(pi) - log(2) - log(sqrt2m)) - sqrt2m;
93         log_det_Sigma = 2 * sum(log(diag(L)));
94         log_const = log(2) - (d / 2) * log(2 * pi) - 0.5 * log_det_Sigma;
95         log_pdf = log_lambda(j) + log_const + log_K;
96     else
97         v = b(j) - d / 2;
98         sqrt2m = sqrt(2 * m);
99         K_v = bessellk(v, sqrt2m);
100        K_v(K_v == Inf) = realmax;
101        K_v(isnan(K_v)) = realmin;
102        K_v(K_v == 0) = realmin;
103        log_K = log(K_v);
104        log_det_Sigma = 2 * sum(log(diag(L)));
105        log_const = log(2) - gammaln(b(j)) - (d / 2) * log(2 * pi) - 0.5 *
            log_det_Sigma + (v) * log(sqrt2m);
106        log_pdf = log_lambda(j) + log_const + log_K;
107    end
108    log_densities(j, :) = log_pdf;
109 end
110 max_log_density = max(log_densities, [], 1);
111 sum_exp = sum(exp(log_densities - max_log_density), 1);
112 ll = sum(max_log_density + log(sum_exp));
113 negloglik = -ll;
114 end
115 function [AIC_matrix, BIC_matrix, AIC_matrixNCT, BIC_matrixNCT] = run_it(
    use_simulated_data)
116 mu_list = {[2; 2], [-2; -2]};
117 Sigma_list = {[1, 0.5; 0.5, 1], [10, -0.3; -0.3, 10]};
118 b_list = [3, 2];
119 if use_simulated_data
120     [data, C] = mixlplacesim(mu_list, Sigma_list, b_list, lambda_true, n);
121 else
122     data = DJIA_stock_selection_highest_cov();

```

```

123     n = size(data, 2);
124 end
125 k = 2;
126 d = 2;
127 initial_params = optimize_initial_values(data, k);
128 lambda_init = initial_params.lambda;
129 gamma_init = log(lambda_init(1:(k-1)) ./ lambda_init(k));
130 mu_init = initial_params.mu;
131 Sigma_list = initial_params.Sigma;
132 cov_params_init = [];
133 for i = 1:k
134     [L, ~] = chol(Sigma_list{i}, 'lower');
135     cov_params_init = [cov_params_init, log(diag(L)')];
136     if d == 2
137         cov_params_init = [cov_params_init, L(2, 1)];
138     end
139 end
140 b_init = initial_params.b;
141 b_init = [1.5, 1.5];
142 param_init = [gamma_init, mu_init(:)', cov_params_init, log(b_init)];
143 options = optimset('Display', 'off', 'MaxIter', 3000, 'TolFun', 1e-10, 'TolX',
144     1e-10);
145 negloglik_fun = @(param) negloglik_laplace(param, data, k);
146 [param_est, fval] = fminunc(negloglik_fun, param_init, options);
147 loglik_laplace = -fval;
148 gamma_est = param_est(1:(k-1));
149 lambda_denominator = 1 + sum(exp(gamma_est));
150 lambda_est = [exp(gamma_est) ./ lambda_denominator, 1 / lambda_denominator];
151 mu_est = reshape(param_est(k:(k + k * d - 1)), [d, k]);
152 cov_params_est = param_est((k + k * d):(k + k * d + k * 3 - 1));
153 b_est = exp(param_est((k + k * d + k * 3):end));
154 Sigma_est_list = cell(k, 1);
155 idx = 1;
156 for i = 1:k
157     L_vec = cov_params_est(idx:(idx + 2));
158     idx = idx + 3;
159     L = [exp(L_vec(1)), 0; L_vec(2), exp(L_vec(3))];
160     Sigma_est_list{i} = L * L';
161 end
162 disp('Estimated Mixing Probabilities (lambda):');
163 disp(lambda_est);
164 disp('Estimated Mean Vectors (mu):');
165 for i = 1:k
166     fprintf('Component %d Mean:\n', i);
167     disp(mu_est(:, i));
168 end
169 disp('Estimated Covariance Matrices (Sigma):');
170 for i = 1:k
171     fprintf('Component %d Covariance Matrix:\n', i);
172     disp(Sigma_est_list{i});
173 end
174 disp(Sigma_est_list{1});
175 disp('Estimated b Values:');
176 for i = 1:k
177     fprintf('Component %d b:\n', i);
178     disp(b_est(i));
179 end
180 disp('Estimated Parameters:');
181 disp(param_est);

```

```

181 num_params_laplace = length(param_est);
182 aic_laplace = 2 * num_params_laplace - 2 * loglik_laplace;
183 bic_laplace = num_params_laplace * log(n) - 2 * loglik_laplace;
184 AIC_matrix = aic_laplace;
185 BIC_matrix = bic_laplace;
186 plot_laplace_contours(data, mu_est, Sigma_est_list, b_est, lambda_est);
187 [param, stderr, iters, loglik2, Varcov] = MVNCT2estimation(data);
188 num_params_NCT = length(param);
189 aic_NCT = 2 * num_params_NCT - 2 * loglik2;
190 bic_NCT = num_params_NCT * log(n) - 2 * loglik2;
191 AIC_matrixNCT = aic_NCT;
192 BIC_matrixNCT = bic_NCT;
193 plot_mvnct_contours(data, param);
194 end
195 function initial_params = optimize_initial_values(data, k)
196 [d, n] = size(data);
197 lambda_init = ones(1, k) / k;
198 [cluster_idx, centroids] = kmeans(data', k, 'Replicates', 10, 'Distance', '
    sqeuclidean');
199 mu_init = centroids';
200 Sigma_list = cell(k, 1);
201 for i = 1:k
202     cluster_data = data(:, cluster_idx == i);
203     if size(cluster_data, 2) > 1
204         Sigma_list{i} = cov(cluster_data');
205     else
206         Sigma_list{i} = eye(d) * 0.1;
207     end
208 end
209 b_init = ones(1, k) * (d + 1)/2;
210 log_b_init = log(b_init);
211 cov_params_init = [];
212 for i = 1:k
213     [L, ~] = chol(Sigma_list{i}, 'lower');
214     cov_params_init = [cov_params_init, log(diag(L)')];
215     if d == 2
216         cov_params_init = [cov_params_init, L(2, 1)];
217     end
218 end
219 gamma_init = log(lambda_init(1:(k-1)) ./ lambda_init(k));
220 param_init = [gamma_init, mu_init(:)', cov_params_init, log_b_init];
221 initial_params = struct();
222 initial_params.lambda = lambda_init;
223 initial_params.mu = mu_init;
224 initial_params.Sigma = Sigma_list;
225 initial_params.b = b_init;
226 initial_params.param_vector = param_init;
227 end
228 function plot_laplace_contours(data, mu_est, Sigma_est_list, b_est, lambda_est)
229 x1_range = linspace(min(data(1, :)) - 1, max(data(1, :)) + 1, 200);
230 x2_range = linspace(min(data(2, :)) - 1, max(data(2, :)) + 1, 200);
231 [X, Y] = meshgrid(x1_range, x2_range);
232 grid_points = [X(:), Y(:)]';
233 pdf_values = zeros(1, size(grid_points, 2));
234 for j = 1:length(mu_est)
235     f = laplacepdf(grid_points, mu_est(:, j), Sigma_est_list{j}, b_est(j));
236     pdf_values = pdf_values + lambda_est(j) * f;
237 end
238 pdf_values = reshape(pdf_values, size(X));

```

```

239 end
240 function plot_mvnct_contours(data, param)
241     v = param(1);
242     mu = param(2:3)';
243     scale = param(4:5)';
244     R12 = param(6);
245     R = [1, R12; R12, 1];
246     Sigma = diag(scale) * R * diag(scale);
247     x1_range = linspace(min(data(1, :)) - 1, max(data(1, :)) + 1, 200);
248     x2_range = linspace(min(data(2, :)) - 1, max(data(2, :)) + 1, 200);
249     [X, Y] = meshgrid(x1_range, x2_range);
250     grid_points = [X(:), Y(:)]';
251     pdf_values = mvnctpdfln(grid_points, mu, param(7:8), v, Sigma);
252     pdf_values = reshape(exp(pdf_values), size(X));
253     hold on;
254     num_contours = 20;
255     contour(X, Y, pdf_values, num_contours, 'LineWidth', 1.5, 'LineColor', [0.3,
        0.8, 0.3]);
256     hold off;
257 end
258 function dataXX = DJIA_stock_selection_highest_cov()
259     fileName = 'DJIA30stockreturns.mat';
260     data = load(fileName);
261     data = data.DJIARet;
262     data(data == 0) = 1e-6;
263     returns = data;
264     covMatrix = cov(returns);
265     covMatrix(logical(eye(size(covMatrix)))) = -Inf;
266     [row, col] = find(triu(covMatrix, 1));
267     covProbs = covMatrix(sub2ind(size(covMatrix), row, col));
268     probs = covProbs / sum(covProbs);
269     selectedIdx = randsample(length(probs), 1, true, probs);
270     pair = [row(selectedIdx), col(selectedIdx)];
271     pairData = data(:, pair);
272     dataX = pairData';
273     dataXX = dataX;
274 end
275 function pdfln = mvnctpdfln(x, mu, gam, v, Sigma)
276     [d, t] = size(x); C = Sigma; [R, err] = cholcov(C, 0);
277     assert(err == 0, 'C is not (semi) positive definite');
278     mu = reshape(mu, length(mu), 1); gam = reshape(gam, length(gam), 1);
279     vn2 = (v + d) / 2; xm = x - repmat(mu, 1, t); rho = sum((R' \ xm).^2, 1);
280     pdfln = gammaln(vn2) - d / 2 * log(pi * v) - gammaln(v / 2) - ...
281         sum(log(diag(R))) - vn2 * log1p(rho / v);
282     if all(gam == 0), return; end
283     idx = (pdfln >= -37); maxiter = 1e4; k = 0;
284     if any(idx)
285         gcg = sum((R' \ gam).^2); pdfln = pdfln - 0.5 * gcg; xcg = xm' * (C \ gam)
286         ;
287         term = 0.5 * log(2) + log(xcg) - 0.5 * log(v + rho');
288         term(term == -inf) = log(realmin); term(term == +inf) = log(realmax);
289         logterms = gammaln((v + d + k) / 2) - gammaln(k + 1) - gammaln(vn2) + k *
290             term;
291         ff = real(exp(logterms)); logsumk = log(ff);
292         while k < maxiter
293             k = k + 1;
294             logterms = gammaln((v + d + k) / 2) - gammaln(k + 1) - gammaln(vn2) +
295                 k * term(idx);
296             ff = real(exp(logterms - logsumk(idx))); logsumk(idx) = logsumk(idx) +

```

```

294         log1p(ff);
295         idx(idx) = (abs(ff) > 1e-4); if all(idx == false), break, end
296     end
297     pdfln = real(pdfln + logsumk');
298 end
299 function [param, stderr, iters, loglik, Varcov] = MVNCT2estimation(data)
300     [d, T] = size(data);
301     assert(d == 2, 'Only supports 2-dimensional data');
302     bound.lo = [1.1, -30, -30, 0.01, 0.01, -1, -4, -4];
303     bound.hi = [100, 30, 30, 100, 100, 1, 4, 4];
304     bound.which = [1, 0, 0, 1, 1, 0, 1, 1];
305     mu_init = mean(data, 2);
306     scale_init = std(data, 0, 2);
307     correlation_init = corr(data(1, :)', data(2, :)'');
308     noncentrality_init = [0.1, 0.1];
309     initvec = [3, mu_init', scale_init', correlation_init, noncentrality_init];
310     initvec = einschraenk(initvec, bound);
311     maxiter = 300;
312     tol = 1e-12;
313     opts = optimset('Display', 'off', 'MaxIter', maxiter, 'TolFun', tol, 'TolX',
314         tol, 'LargeScale', 'off');
315     obj_fun = @(param) MVNCTloglik(param, data, bound);
316     [pout, fval, ~, output, ~, hess] = fminunc(obj_fun, initvec, opts);
317     [param, Varcov] = einschraenk(pout, bound, inv(hess) / T);
318     stderr = sqrt(diag(Varcov));
319     loglik = -fval * T;
320     iters = output.iterations;
321 end
322 function ll = MVNCTloglik(param, x, bound)
323     if nargin < 3, bound = 0; end
324     if isstruct(bound), param = einschraenk(real(param), bound, 999); end
325     k = param(1); mu = param(2:3); scale = param(4:5); gam = param(7:8);
326     R12 = param(6); R = [1, R12; R12, 1];
327     if min(eig(R)) < 1e-4
328         ll = 1e5;
329     else
330         xx = x;
331         for i = 1:2, xx(i, :) = (x(i, :) - mu(i)) / scale(i); end
332         llvec = mvnctpdfln(xx, mu, gam, k, R) - log(prod(scale));
333         ll = -mean(llvec);
334         if isinf(ll), ll = 1e5; end
335     end
336 end
337 function [y, V_out] = einschraenk(x, bound, V_in)
338     y = x;
339     for i = 1:length(x)
340         if y(i) < bound.lo(i)
341             y(i) = bound.lo(i);
342         elseif y(i) > bound.hi(i)
343             y(i) = bound.hi(i);
344         end
345     end
346     if nargin == 3 && ~isempty(V_in)
347         V_out = V_in;
348         for i = 1:length(y)
349             if bound.which(i) == 0
350                 V_out(i, :) = 0;
351                 V_out(:, i) = 0;

```

```
351         end
352     end
353 else
354     V_out = [];
355 end
356 end
```